AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

DEEP LEARNING · PROJECT REVIEW REPORT

# Swin Transformer Image Captioning on ROCOv2

Hierarchical Shifted-Window Vision Transformer and Causal Language Decoding for Automated Radiology Report Generation


SUBMITTED BY

| | |
|---|---|
| **Dokku Naga Shiva** | CB.SC.U4AIE24315 |
| **Rajashree T** | CB.SC.U4AIE24346 |
| **Ravishanmugam K** | CB.SC.U4AIE24347 |
| **Sharvesh Sivagnanam** | CB.SC.U4AIE24355 |

UNDER THE GUIDANCE OF

**Dr. Mithun Kumar Kar**

Assistant Professor, Department of Artificial Intelligence



Amrita Vishwa Vidyapeetham, Coimbatore · October 2026

# Abstract

This report documents the design, mathematical formulation, implementation, and empirical benchmark of an end-to-end medical image captioning system built on a hierarchical **Swin Transformer (Swin-T)** visual encoder and a **6-layer causal Transformer language decoder**. The system autoregressively generates structured diagnostic descriptions directly from diverse radiology modalities (including computed tomography, chest radiographs, magnetic resonance imaging, and angiography) on the large-scale **ROCOv2** benchmark (59,958 training, 9,904 validation, and 9,927 held-out test image–caption pairs).

The proposed framework resolves the classical single-vector spatial bottleneck inherent in convolutional recurrent architectures by preserving a $7 \times 7$ lattice of **49 spatially grounded visual memory tokens** ($Z \in \mathbb{R}^{B\times 49\times 768}$) through local and shifted window multi-head self-attention (W-MSA / SW-MSA). The 6-layer Transformer decoder dynamically attends to these spatial tokens via multi-head cross-attention at every token prediction step. The network is optimized using a **two-phase transfer learning schedule** (frozen encoder warmup followed by differential end-to-end fine-tuning) coupled with word dropout ($p = 0.10$), decoder dropout ($0.20$), and label smoothing ($\epsilon = 0.10$) to prevent autoregressive mode collapse.

Evaluated on the full **9,927 test split** under identical runtime conditions on an NVIDIA GeForce RTX 4060 GPU, the Swin Transformer outperforms the classical ResNet-50 + LSTM baseline across all quantitative n-gram precision, structural recall, and consensus metrics. Under greedy decoding, our model achieves a BLEU-1 of **0.1716** (vs. 0.1571 for the baseline), BLEU-4 of **0.0327** (vs. 0.0273, a $+19.8\%$ relative improvement), ROUGE-L of **0.2033** (vs. 0.1831), and CIDEr-D of **0.2171** (vs. 0.1726, a $+25.8\%$ gain). When augmented with length-normalized beam search ($k = 4$), performance further advances to **BLEU-1 0.1833**, **BLEU-4 0.0379**, **ROUGE-L 0.2160**, and **CIDEr-D 0.3438**, while elevating the proportion of unique, non-repetitive descriptions from $43.98\%$ to **79.80% distinct captions**. While the recurrent baseline exhibits lower per-image compute latency (7.98 ms vs. 40.52 ms), the hierarchical Swin Transformer successfully mitigates generic modal collapse and delivers clinically grounded report generation suitable for offline radiology drafting.

**Keywords:** Medical Image Captioning, Swin Transformer, Vision Transformer, Multi-Head Cross-Attention, ROCOv2, Transfer Learning, Beam Search Decoding.

# Contents

**Abstract** **2**

**1 Introduction** **5**
1.1 Problem and motivation . . . 5
1.2 Challenges of medical captioning . . . 5
1.3 Related work . . . 6
1.4 Objectives . . . 6

**2 Dataset and Preprocessing** **7**
2.1 The ROCOv2 dataset . . . 7
2.2 Image preprocessing . . . 7
2.3 Caption preprocessing . . . 8

**3 Methodology** **10**
3.1 Overview: the pipeline in eight steps . . . 10
3.2 Attention: the core operation . . . 11
3.3 Step 1 — Image to Patch Tokens . . . 13
3.4 Step 2 — Swin Transformer Block Architecture . . . 14
3.5 Step 3 — Window Attention (W-MSA) and Complexity . . . 15
3.6 Step 4 — Shifted Window Attention (SW-MSA) and Cyclic Shifting . . . 16
3.7 Step 5 — Hierarchical Patch Merging and Image Memory . . . 17
3.8 Step 6 — Caption Token Representation and Input Conditioning . . . 18
3.9 Step 7 — 6-Layer Transformer Decoder Architecture . . . 19
3.10 Step 8 — Autoregressive Decoding and Search Strategies . . . 21
3.11 Training . . . 23
3.12 Summary: the four attention types . . . 26

**4 Baseline Architecture: ResNet-50 + LSTM** **27**
4.1 Architectural Pipeline . . . 27
4.2 Mathematical Formulation . . . 27

**5 Evaluation Metrics** **29**
5.1 BLEU-1 to BLEU-4 . . . 29
5.2 ROUGE-L . . . 29
5.3 CIDEr-D . . . 30
5.4 METEOR . . . 30
5.5 Distinct captions . . . 30

**6 Experimental Results and Comparative Analysis** **32**
6.1 Training Dynamics and Convergence Behavior . . . 32
6.2 Head-to-Head Test Split Benchmark (9,927 Images) . . . 32

6.3 Computational Resources, Memory Footprint, and Latency Analysis . . . . . 34
6.4 Token Classification and Diagnostic Confusion Analysis . . . . . 35
6.5 Qualitative Inferences and Comparative Case Study . . . . . 36

**7 Ablation Studies and Diagnostic Investigations** **38**
7.1 Effect of Training Scale and Transfer Learning . . . . . 38
7.2 Verification of Optimization Capability (10-Example Overfit Test) . . . . . 39
7.3 Visual Feature Activation and Spatial Localization . . . . . 39
7.4 Clinical Failure Mode Taxonomy . . . . . 39

**8 Discussion, Limitations, and Future Directions** **41**
8.1 Discussion of Architectural and Empirical Findings . . . . . 41
8.2 System Limitations . . . . . 41
8.3 Future Research Directions . . . . . 42

**9 Conclusion** **43**

**References** **44**

# 1 Introduction

## 1.1 Problem and motivation

Every radiology image has to be described in words by a trained radiologist, and writing these descriptions takes time. **Image captioning** means generating such a description automatically: the model receives the pixels of an image and returns a sentence. An automatic first draft can save reporting effort and make descriptions more consistent.

The model works in two steps, like a radiologist who first studies the scan and then dictates one sentence: an **encoder** *looks* at the image, and a **decoder** *writes* the caption one word at a time (Figure 1.1).

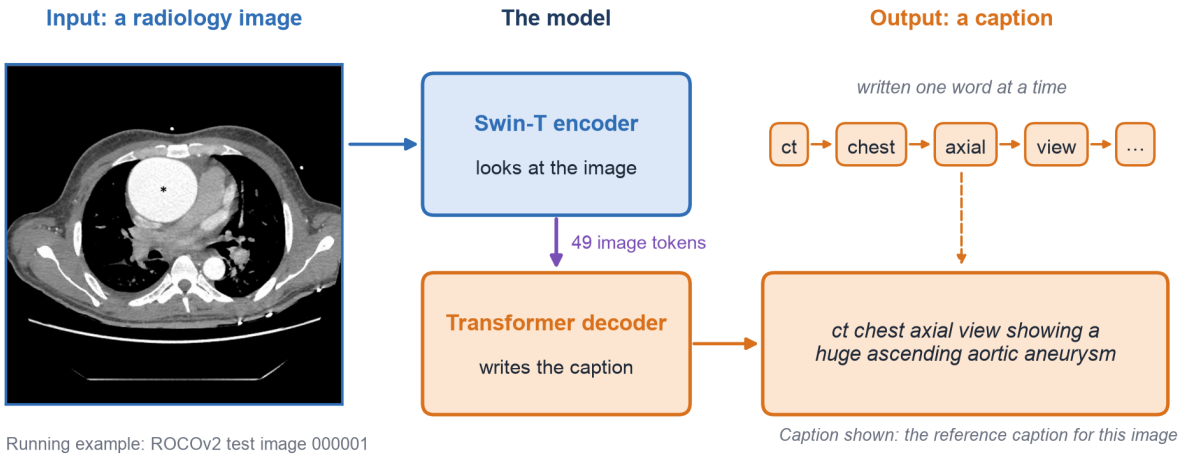



**Figure 1.1:** The captioning task. The Swin-T encoder turns the scan into 49 image tokens; the Transformer decoder uses them to write the caption one word at a time. This scan, ROCOv2 test image 000001, is the running example of the report.

Writing one word at a time follows from the chain rule of probability. The probability of a caption $y = (y_1, \dots, y_T)$ for an image $I$ is the product of the probabilities of its words, each given the image and the words before it:

$$p(y|I) = \prod_{t=1}^{T} p\left(y_t | y_1, \dots, y_{t-1}, I\right) \tag{1.1}$$

For example, if $p(\text{ct}|I) = 0.6$ and $p(\text{chest}|\text{ct}, I) = 0.5$, then the caption "ct chest" (before the end token) has probability $0.6 \times 0.5 = 0.3$.

## 1.2 Challenges of medical captioning

- **Small findings.** The important region is often a small part of the scan, so the model must keep track of *where* things are.
- **Specialised vocabulary.** Words such as *pseudoaneurysm* or *extravasation* are rare. Even after rare words are removed, the vocabulary has 8,729 tokens.

- **Captions are figure legends.** ROCOv2 captions come from journal articles. They refer to arrows and panel letters, and some run to more than 100 words.
- **Generic captions.** A model can score reasonably by always writing something safe, such as "computed tomography scan of the chest showing a mass". The share of *distinct* captions has to be checked as well.

## 1.3 Related work

**Table 1.1:** Earlier work this project builds on.

| Work | Key idea | What we use from it |
|---|---|---|
| Show and Tell [1] | CNN encodes the image as one vector; an LSTM writes the caption | Design of the CNN + LSTM baseline |
| Show, Attend and Tell [2] | The decoder looks at a grid of image regions for each word | The idea behind cross-attention |
| Transformer [3] | Attention instead of recurrence; masked self-attention, cross-attention | The decoder |
| ViT [4] | An image becomes a sequence of patches | Patch embedding |
| Swin Transformer [5] | Window attention, shifted windows, patch merging, pyramid | The encoder |
| ROCO [6], ROCOv2 [7] | Radiology image–caption pairs from PubMed Central | The dataset |
| Lee et al. [8] | Transformer captioning of medical images with global and local features | Closest related system |
| Pranay Kumar et al. [9] | CNN + LSTM caption generator | Reference for the baseline |

## 1.4 Objectives

1. Build a radiology captioning model from a **Swin-T encoder** and a **6-layer Transformer decoder**.
2. Explain each component with intuition, equations, diagrams and small worked examples.
3. Train it on ROCOv2 with **transfer learning** and a **two-phase** schedule.
4. Evaluate it on the full test split (BLEU, ROUGE-L, CIDEr-D, METEOR, distinct captions) and compare it with a **CNN + LSTM baseline**.

**How to read this report.** Chapter 2 prepares the data. Chapter 3 follows one image and one caption through the model, step by step. Chapters 4–9 cover the baseline, metrics, results and conclusions. In every figure, blue marks the image and encoder, orange the words and decoder, purple attention, green MLP layers, and red masked positions. Figures marked *illustration* use made-up numbers to explain an idea.

# 2 Dataset and Preprocessing

## 2.1 The ROCOv2 dataset

ROCOv2 [7] contains 79,789 radiology images (CT, MRI, X-ray, ultrasound, angiography and others) taken from open-access articles on PubMed Central. Each image comes with its figure legend as the caption. We use the official splits (Figure 2.1):

- **Training set:** updates the weights.
- **Validation set:** chooses the best epoch.
- **Test set:** used only once, for the final scores.

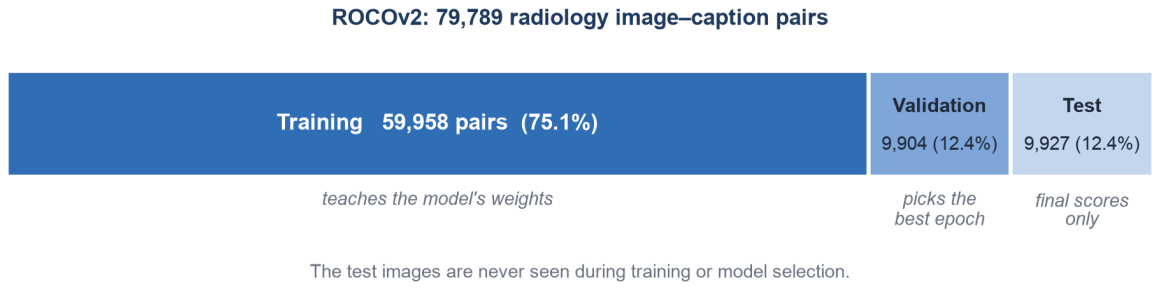



**Figure 2.1:** The ROCOv2 splits.

The data is very varied, as Figure 2.2 shows. Captions range from 6 words to more than 100, and many refer to marks drawn on the figure, such as "(*)" or "(white arrow)".

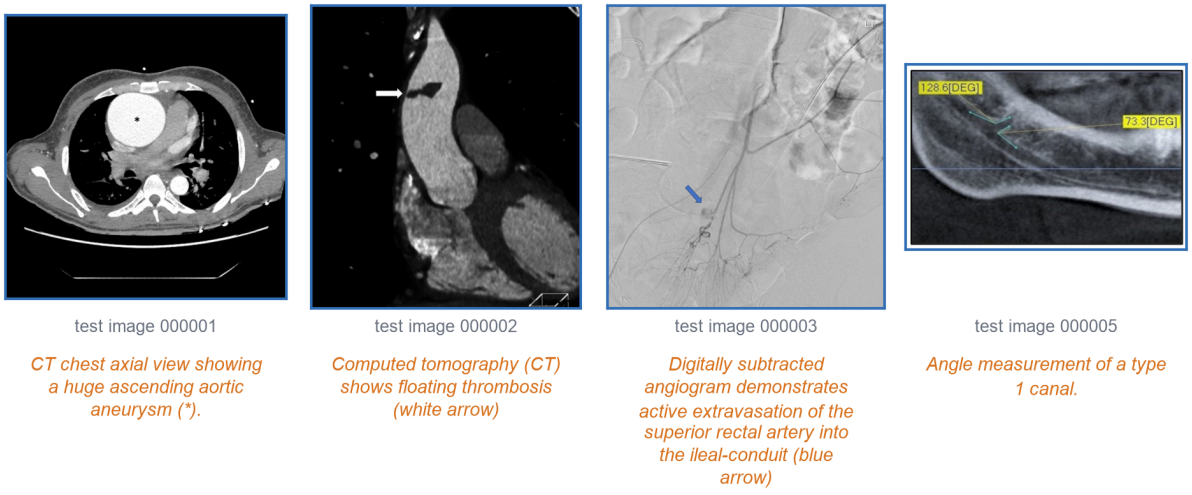



**Figure 2.2:** Four ROCOv2 test images with their reference captions.

## 2.2 Image preprocessing

Every scan becomes a $3 \times 224 \times 224$ tensor (Figure 2.3):

1. **Resize** to $224 \times 224$ pixels.

2. **Augment** (training images only): random horizontal flip, rotation of up to $\pm 10°$, shift of up to 5%, and brightness or contrast change of up to 10%.
3. **Normalise** each colour channel with the ImageNet statistics, because the encoder was pretrained on ImageNet [10].

Grey scans are copied into all three channels.

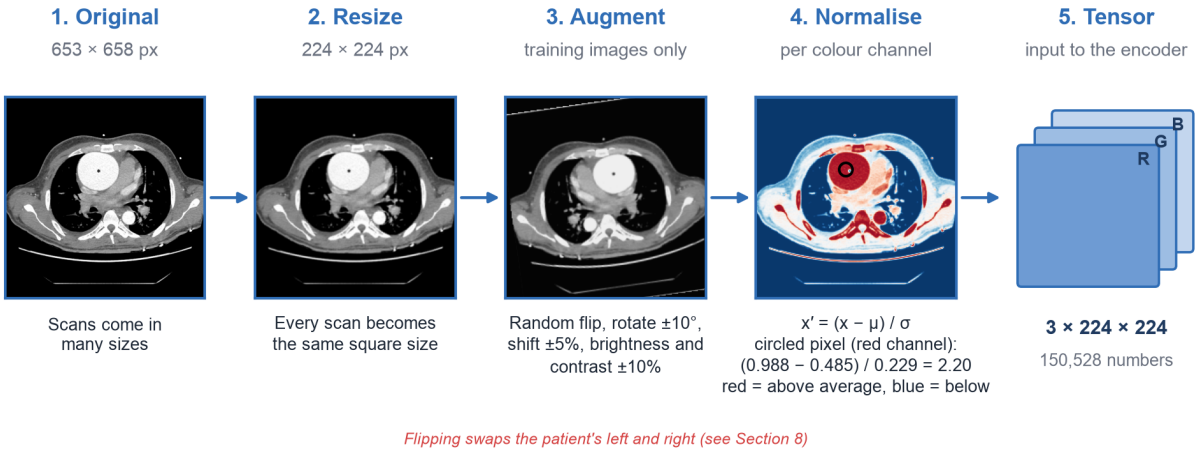



**Figure 2.3:** The image pipeline on the running example. Augmentation (step 3) is applied only during training.

Each pixel value $p$ (0–255) is first scaled to $x = p/255$. Then, for each colour channel $c$:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}, \qquad \mu = (0.485,\ 0.456,\ 0.406),\ \ \sigma = (0.229,\ 0.224,\ 0.225) \tag{2.1}$$

> **Worked example.** The circled pixel inside the aneurysm has $p = 252$, so $x = 0.988$. In the red channel this becomes $(0.988 - 0.485)/0.229 = 2.20$. A black background pixel becomes $(0 - 0.485)/0.229 = -2.12$.

## 2.3 Caption preprocessing

Each caption becomes a list of exactly 40 integers (Figure 2.4), in four steps:

1. **Clean:** lower-case the text and remove every character that is not a–z or 0–9.
2. **Split:** cut the text into words.
3. **Map to IDs:** replace each word by its position in the **vocabulary**. The vocabulary holds the 4 special tokens plus every word seen at least 5 times in the training captions: 8,729 tokens in total.
4. **Add start, end and padding:** put `<sos>` in front and `<eos>` at the end, then pad with `<pad>` up to 40 tokens.

The special tokens are `<pad>` (ID 0), `<sos>` (1), `<eos>` (2) and `<unk>` (3); any word not in the vocabulary becomes `<unk>`.

With words $w_1, \ldots, w_n$ and $T = 40$, the encoded caption is

$$y = [\,1,\ \mathrm{id}(w_1),\ \ldots,\ \mathrm{id}(w_m),\ 2,\ 0,\ \ldots,\ 0\,], \qquad m = \min(n,\ 38) \tag{2.2}$$

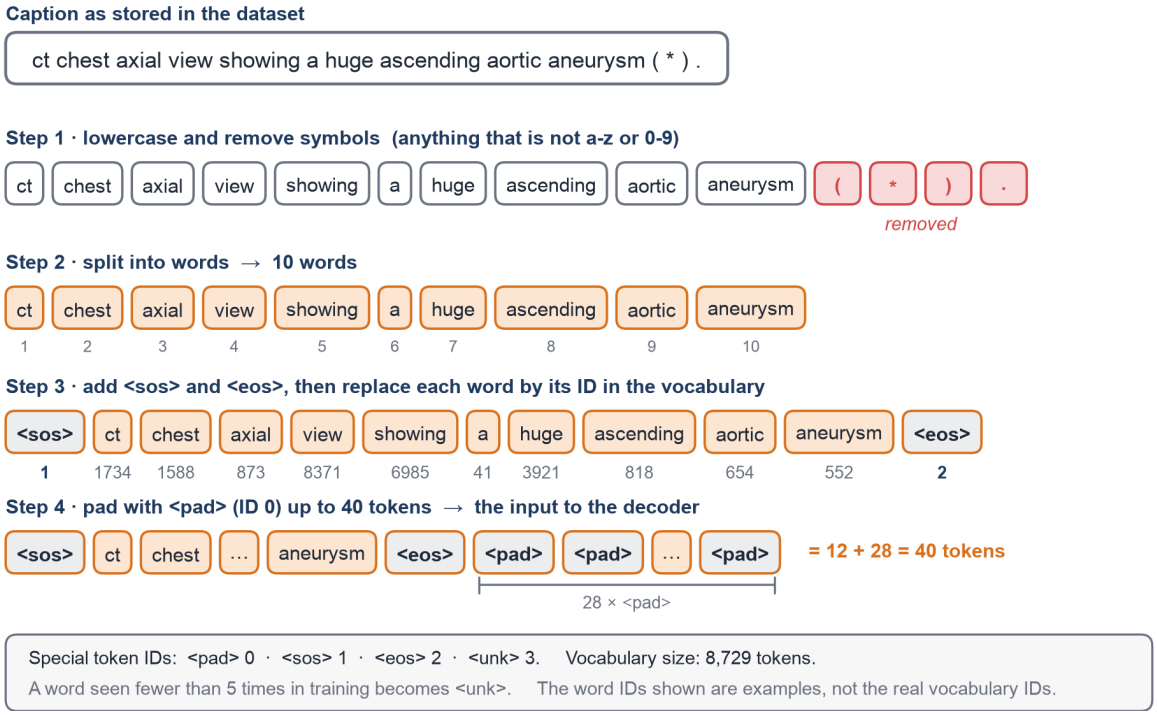



**Figure 2.4:** The caption pipeline on the running example. Ten words plus `<sos>` and `<eos>` give 12 tokens, followed by 28 padding tokens.

so at most 38 words are kept. One training batch therefore holds an image tensor of shape $(32, 3, 224, 224)$ and a caption tensor of shape $(32, 40)$.

# 3 Methodology

## 3.1 Overview: the pipeline in eight steps

This chapter follows the running example, one chest CT and its caption, through the model in the order the data flows (Figure 3.1).

**Encoder (Steps 1–5):** turns the image into 49 summary vectors.

1. **Image → patch tokens:** the scan is cut into small patches, and each patch becomes a vector (3.3).
2. **Swin block:** LayerNorm, attention and an MLP update the tokens (3.4).
3. **Window attention:** tokens exchange information inside $7 \times 7$ windows (3.5).
4. **Shifted windows:** every second block shifts the windows so neighbouring windows connect (3.6).
5. **Patch merging and four stages:** the grid of tokens shrinks from $56 \times 56$ to $7 \times 7$, giving the **image memory** $Z$ (3.7).

**Decoder (Steps 6–8):** turns $Z$ into a caption.

6. **Caption → decoder input:** word embedding, positional encoding and teacher forcing (3.8).
7. **Decoder layers:** masked self-attention reads the earlier words; cross-attention reads the image (3.9).
8. **Output and generation:** softmax gives next-word probabilities; beam search chooses the caption (3.10).

**Training** follows in Section 3.11. Attention is used in Steps 3, 4 and 7, so Section 3.2 explains it first.

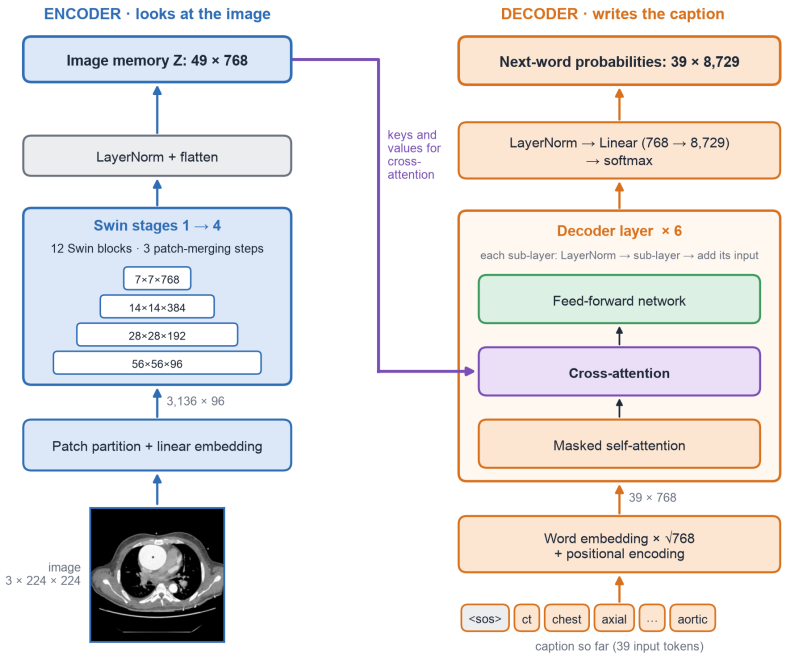



**Figure 3.1:** The full model. Left: the Swin-T encoder turns the image into the image memory Z. Right: the Transformer decoder turns the caption written so far into next-word probabilities, reading Z through cross-attention in each of its six layers.

**Table 3.1:** Notation and sizes used in this chapter.

| Symbol | Meaning | Value in our model |
|---|---|---|
| $B$ | Batch size | 32 |
| $H_s \times H_s$ | Token grid at encoder stage $s$ | $56^2,\ 28^2,\ 14^2,\ 7^2$ |
| $C_s$ | Numbers per token at stage $s$ | 96, 192, 384, 768 |
| $M$ | Window size (tokens per side) | 7 |
| $h$, $d$ | Attention heads, numbers per head | encoder: 3–24 heads $\times$ 32; decoder: 8 heads $\times$ 96 |
| $Z$ | Image memory (encoder output) | $49 \times 768$ |
| $T$ | Caption length in tokens | 40 (39 decoder positions) |
| $\lvert V\rvert$, $E$ | Vocabulary size, embedding table | 8,729; $8{,}729 \times 768$ |

## 3.2 Attention: the core operation

**What it does.** Attention lets each token update itself using information from other tokens. Every token produces three vectors:

- a **query** ($q$): "what am I looking for?";
- a **key** ($k$): "what do I contain?";
- a **value** ($v$): "what do I pass on?".

A token compares its query with every key. The better the match, the more of that token's value it takes, so the result is a weighted average of the values.

> **Everyday comparison: a library search.** Your question is the query, the titles on the book spines are the keys, and the contents of the books are the values. You read mostly the books whose titles match your question best.

**The math.** For input tokens $X$ (and, in cross-attention, a second set of tokens $X'$ that supplies the keys and values):

$$Q = XW_Q, \qquad K = X'W_K, \qquad V = X'W_V \tag{3.1}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B\right)V \tag{3.2}$$

where:

- $W_Q$, $W_K$ and $W_V$ are learned matrices;
- $d$ is the number of numbers per head;
- $B$ is an optional term: a position bias, or a mask that blocks some pairs;
- softmax turns each row of scores into positive weights that sum to 1.

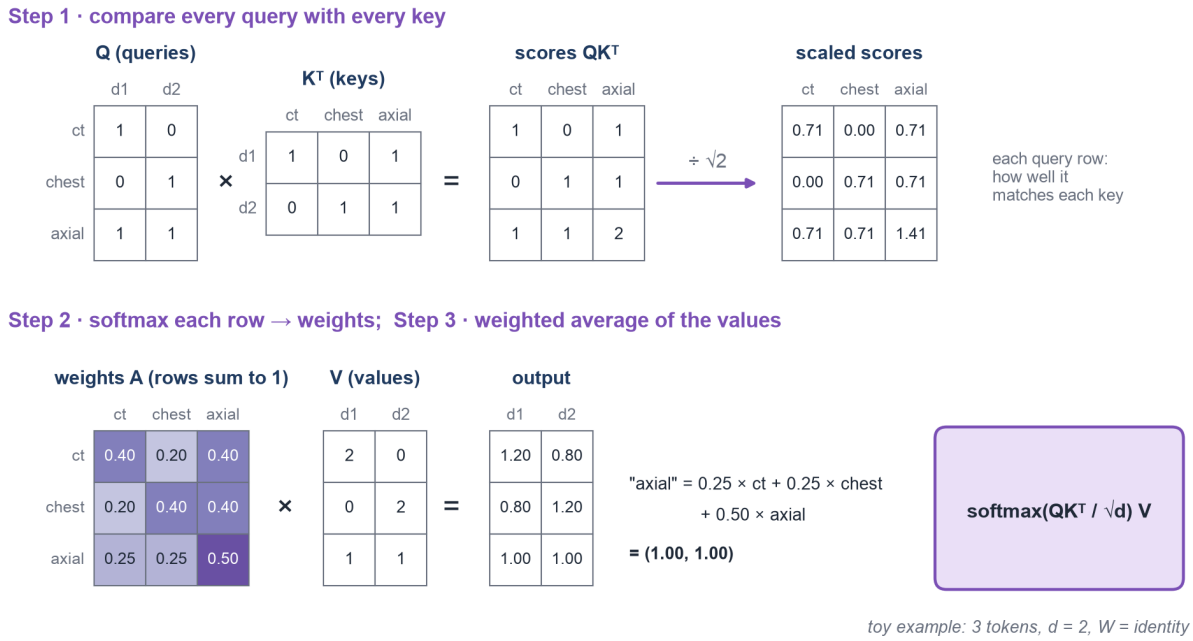



**Figure 3.2:** Attention computed step by step for three tokens with d = 2. Step 1 compares queries with keys; step 2 turns each row into weights with softmax; step 3 averages the values with those weights.

> **Worked example (Figure 3.2).** The query of "axial" is $(1, 1)$.
> **Scores.** Compared with the keys of (ct, chest, axial), it scores $(1, 1, 2)$, or $(0.71, 0.71, 1.41)$ after dividing by $\sqrt{2}$.
> **Weights.** Softmax gives $(0.25, 0.25, 0.50)$.
> **Output.** $0.25 \times (2, 0) + 0.25 \times (0, 2) + 0.50 \times (1, 1) = (1.00,\ 1.00)$: a blend of all three values that leans towards "axial" itself.

**Why divide by $\sqrt{d}$?** A dot product of two random $d$-number vectors grows roughly like $\sqrt{d}$: about 1.4 for $d = 2$ but 9.8 for $d = 96$. Without scaling, softmax would give almost all the weight to one key. For example, the scores $(1.0, 0.5, 0.2, -0.3)$ multiplied by 9.8 give weights $(0.99, 0.01, 0.00, 0.00)$, while the scaled scores give $(0.43, 0.26, 0.19, 0.12)$.

**Multi-head attention.** The $C$ numbers of each token are split into $h$ **heads** of $d = C/h$ numbers. Each head computes its own attention, so it can look for a different kind of relationship. The heads' results are joined and mixed by a final linear layer:

$$\mathrm{MHA}(X) = [\mathrm{head}_1;\ \ldots;\ \mathrm{head}_h]\, W_O \tag{3.3}$$

Attention is used in four ways in this model (Figure 3.3). Each is worked through with numbers where it first appears:

- **Window self-attention:** Section 3.5.
- **Shifted-window attention:** Section 3.6.
- **Masked self-attention and cross-attention:** Section 3.9.

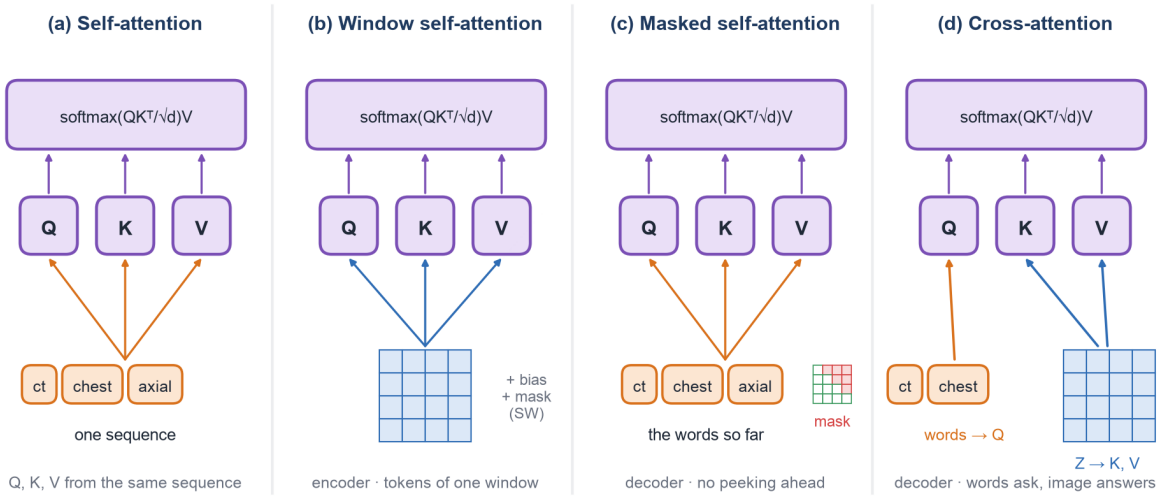



**Figure 3.3:** The four types of attention in the model, and where their queries, keys and values come from.

## 3.3 Step 1 — Image to Patch Tokens

**Patch Partitioning and Linear Projection.** The raw radiograph $X \in \mathbb{R}^{B\times 3\times 224\times 224}$ is partitioned into non-overlapping spatial patches of size $P \times P$ pixels ($P = 4$). Each patch spans $4 \times 4 \times 3 = 48$ raw pixel channels, yielding a grid of $\frac{H}{P} \times \frac{W}{P} = 56 \times 56 = 3{,}136$ patches. A shared linear projection layer maps each flattened 48-dimensional patch into an embedding space of dimension $C = 96$, followed by Layer Normalization:

$$z_{ij} = \mathrm{LN}\left(W_p\,\mathrm{vec}(x_{ij}) + b_p\right), \qquad W_p \in \mathbb{R}^{96\times 48}, \quad b_p \in \mathbb{R}^{96} \tag{3.4}$$

In convolutional terms, this linear projection is mathematically equivalent to a 2D convolution with 96 kernels of size $4 \times 4$, applied with stride $S = 4$ and zero padding.

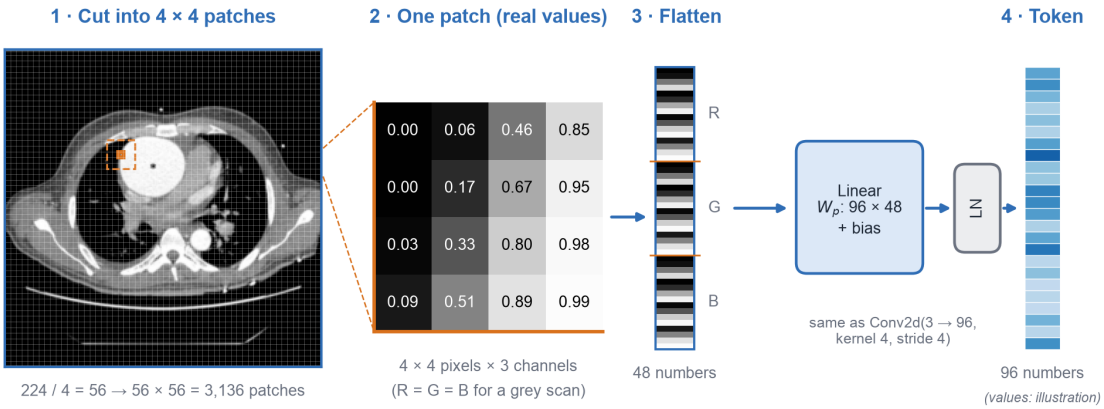



**Figure 3.4:** Patch embedding pipeline. The $224 \times 224$ radiograph is partitioned into a $56 \times 56$ lattice of $4 \times 4$ patches (48 values each), projected linearly to dimension $C = 96$, and normalized via LayerNorm.

**DL Architectural Design Choice.** Treating pixels directly as tokens would yield 50,176 sequence positions, inducing an intractable attention memory footprint. Patch partitioning reduces the sequence length by a factor of $P^2 = 16$ (3,136 tokens) while preserving local sub-anatomical structures.

## 3.4 Step 2 — Swin Transformer Block Architecture

The token sequence passes through 12 Swin Transformer blocks distributed across 4 stages ($[2, 2, 6, 2]$). Every block employs a pre-LayerNorm (Pre-LN) residual architecture:

$$\hat{z}^l = \text{W-MSA}\left(\text{LN}\left(z^{l-1}\right)\right) + z^{l-1}, \qquad z^l = \text{MLP}\left(\text{LN}\left(\hat{z}^l\right)\right) + \hat{z}^l \tag{3.5}$$

$$\hat{z}^{l+1} = \text{SW-MSA}\left(\text{LN}\left(z^l\right)\right) + z^l, \qquad z^{l+1} = \text{MLP}\left(\text{LN}\left(\hat{z}^{l+1}\right)\right) + \hat{z}^{l+1} \tag{3.6}$$

where W-MSA denotes Window-based Multi-Head Self-Attention and SW-MSA denotes Shifted-Window Multi-Head Self-Attention.

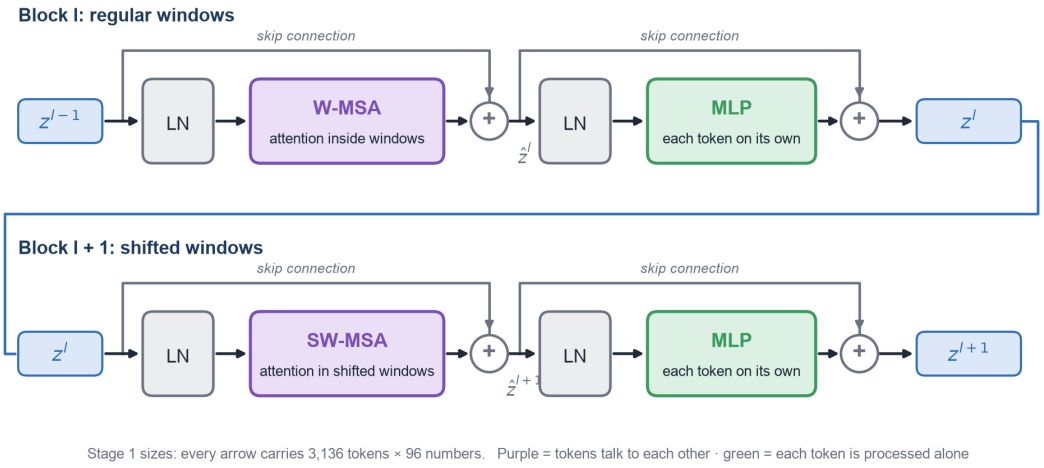



**Figure 3.5:** Consecutive Swin Transformer block pair. Block $l$ applies local window self-attention (W-MSA), and block $l + 1$ applies shifted-window self-attention (SW-MSA). Both utilize pre-LayerNorm and residual skip connections.

**Sub-Layer Components:**

- **Layer Normalization (LN):** Normalizes activations across channel dimensions independently per token, ensuring numerical stability across deep feature hierarchies:

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i \tag{3.7}$$

- **Multilayer Perceptron (MLP):** Applies two affine transformations with an intermediate GELU activation, expanding feature width by an expansion ratio of 4 ($C \to 4C \to C$):

$$\text{MLP}(x) = W_2\,\text{GELU}(W_1 x + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{4C \times C}, \quad W_2 \in \mathbb{R}^{C \times 4C} \tag{3.8}$$

- **Stochastic Depth (DropPath):** Linearly escalates drop probability ($0 \rightarrow 0.20$) across blocks, randomly bypassing residual branches during training to act as an implicit ensemble regularizer.

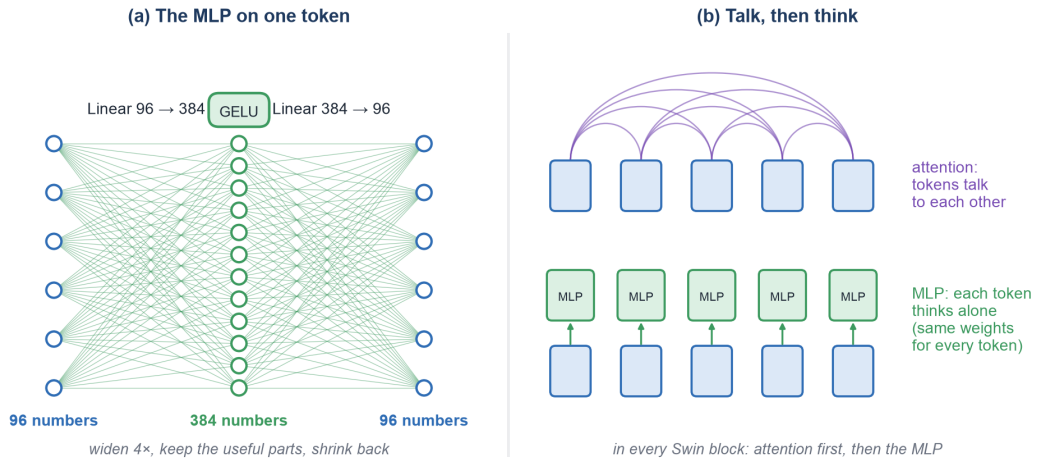



**Figure 3.6:** Position-wise Feed-Forward Network (MLP): (a) Expands channel capacity by $4\times$ ($96 \rightarrow 384$) before non-linear projection via GELU. (b) Executes independently per spatial token.

## 3.5 Step 3 — Window Attention (W-MSA) and Complexity

**Local Window Partitioning.** Standard global Vision Transformer attention scales quadratically with sequence length ($\mathcal{O}(N^2)$), rendering high-resolution medical imaging computationally prohibitive. Swin Transformer resolves this by partitioning the $H \times W$ token lattice into non-overlapping windows of size $M \times M$ ($M = 7$). At Stage 1, this creates $\frac{56}{7} \times \frac{56}{7} = 64$ distinct windows, each containing $M^2 = 49$ tokens.

Self-attention is executed strictly within each window:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + B\right)V \tag{3.9}$$

where $Q, K, V \in \mathbb{R}^{M^2 \times d_k}$ and $B \in \mathbb{R}^{M^2 \times M^2}$ is a learned **continuous relative position bias**. Since spatial coordinates within an $M \times M$ window vary in range $[-M+1, M-1]$, bias values are dynamically indexed from a compact parameter table $\hat{B} \in \mathbb{R}^{(2M-1)\times(2M-1)}$:

$$B_{ij} = \hat{B}\left[\Delta r_{ij} + M - 1,\ \Delta c_{ij} + M - 1\right] \tag{3.1}$$

**Computational Complexity Comparison.** For a feature map of $h \times w$ patches with $C$ channels:

$$\Omega(\text{Global MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC \tag{3.10}$$

Because $M = 7$ is fixed, W-MSA scales **linearly** with image area $hw$. At Stage 1 ($56 \times 56, C = 96$), W-MSA requires $\approx 0.15$ GFLOPs compared to $\approx 2.00$ GFLOPs for global attention—a $13.3\times$ computational saving.

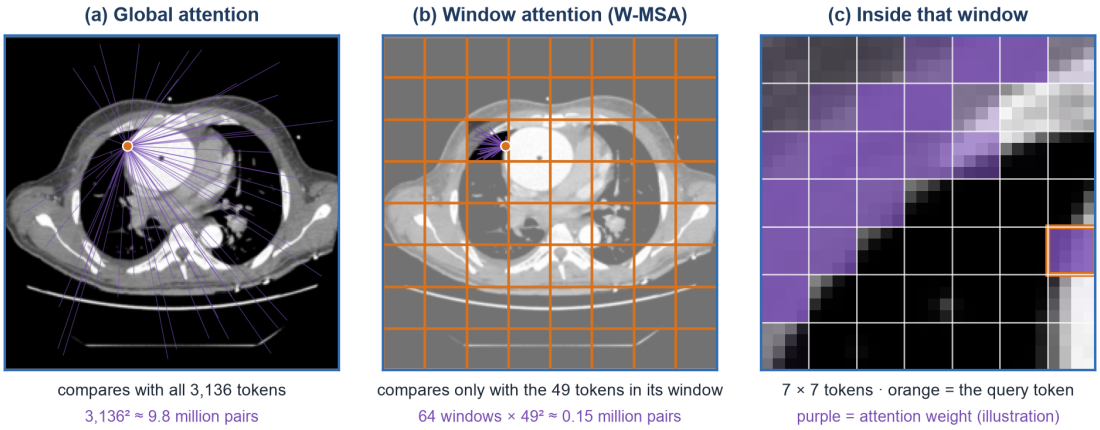



**Figure 3.7:** Comparison of global attention vs. local Window-based attention (W-MSA). W-MSA confines pairwise token interactions to $7 \times 7$ spatial windows, bounding quadratic scaling.

## 3.6 Step 4 — Shifted Window Attention (SW-MSA) and Cyclic Shifting

**Cross-Window Information Flow.** While W-MSA achieves linear complexity, restricting attention inside static windows eliminates inter-window receptive field growth. To restore cross-window connections without incurring global quadratic cost, consecutive blocks shift the partitioning grid by $\lfloor M/2 \rfloor = 3$ tokens along both spatial axes.

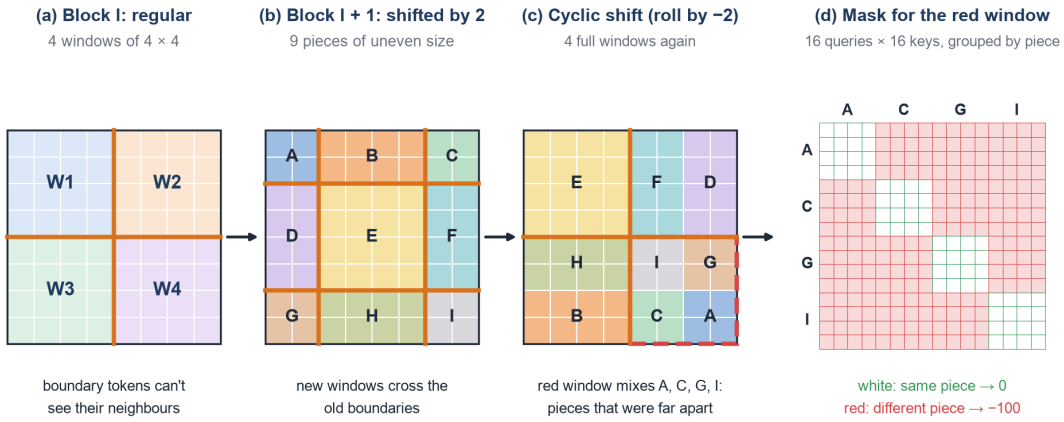



**Figure 3.8:** Efficient shifted-window computation via cyclic rolling and attention masking. (a) Standard window partition. (b) Shifted windows create sub-window fragments. (c) Cyclic shift merges fragments into regular $M \times M$ windows. (d) Self-attention masking blocks cross-fragment attention.

**Cyclic Shift Formulation.** A naive shift produces $2 \times 2 = 4$ window fragments at borders and corners, increasing the window count from $\lceil \frac{h}{M} \rceil \times \lceil \frac{w}{M} \rceil$ to $(\lceil \frac{h}{M} \rceil + 1) \times (\lceil \frac{w}{M} \rceil + 1)$. Swin Transformer avoids this via **cyclic rolling**:

$$X'[r, c] = X\left[(r + \lfloor M/2 \rfloor) \bmod H,\ (c + \lfloor M/2 \rfloor) \bmod W\right] \tag{3.11}$$

After cyclic shifting, batched window attention is executed with an additive attention mask $\Phi$:

$$\text{SW-MSA} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + B + \Phi\right) V, \qquad \Phi_{ij} = \begin{cases} 0, & \text{tokens } i, j \text{ from same source region} \\ -\infty, & \text{tokens } i, j \text{ from disparate regions} \end{cases} \tag{3.12}$$

After attention, a reverse cyclic shift restores tokens to their original geometric positions.

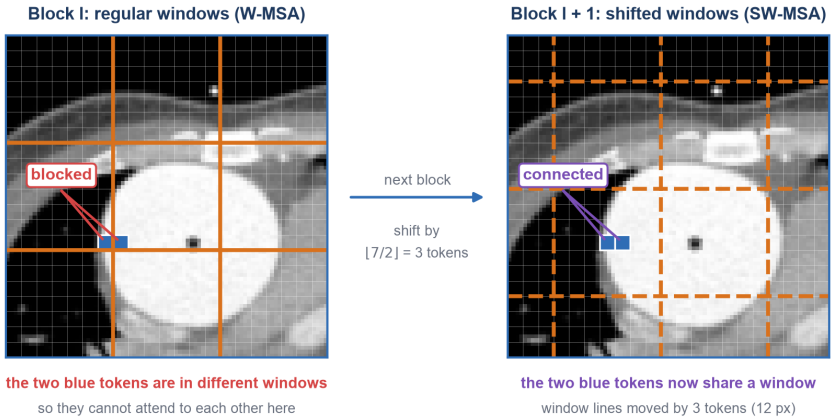



**Figure 3.9:** Shifted-window receptive field bridging across consecutive blocks on an anatomical scan. Tokens belonging to adjacent partitions in block $l$ share joint attention in block $l + 1$.

## 3.7 Step 5 — Hierarchical Patch Merging and Image Memory

**Spatial Downsampling and Channel Expansion.** To construct a multiscale feature pyramid analogous to a CNN, **Patch Merging** layers are inserted between stages. Each patch merging layer concatenates $2 \times 2$ spatially neighboring tokens ($4C$ channels), normalizes them via LayerNorm, and applies a linear projection to reduce channels to $2C$:

$$z'_{ij} = W_m \,\text{LN}\left(\left[x_{2i,2j} \,\|\, x_{2i+1,2j} \,\|\, x_{2i,2j+1} \,\|\, x_{2i+1,2j+1}\right]\right), \qquad W_m \in \mathbb{R}^{2C \times 4C} \tag{3.13}$$

This operation halves spatial resolution while doubling channel capacity across the 4 hierarchy stages (Table 3.1).

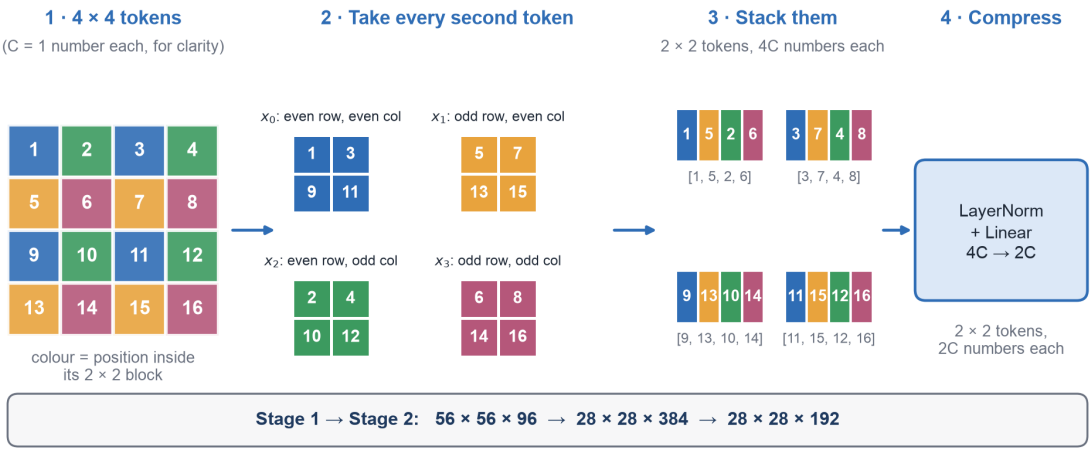



**Figure 3.10:** Patch Merging operation. Spatial tokens within each $2 \times 2$ neighborhood are concatenated ($4C$ channels) and linearly projected to $2C$, reducing spatial dimension by $2\times$.

**Table 3.2:** Architectural parameters across the four hierarchical stages of Swin-T.

| Stage | Spatial Lattice | Channels ($C$) | Blocks | Heads | Windows ($n_W$) | Patch Field |
|---|---|---|---|---|---|---|
| 1 | $56 \times 56$ | 96 | 2 | 3 | 64 | $4 \times 4$ px |
| 2 | $28 \times 28$ | 192 | 2 | 6 | 16 | $8 \times 8$ px |
| 3 | $14 \times 14$ | 384 | 6 | 12 | 4 | $16 \times 16$ px |
| 4 | $7 \times 7$ | 768 | 2 | 24 | 1 | $32 \times 32$ px |

**Extraction of the Spatial Image Memory ($Z$).** Following Stage 4 and a final LayerNorm, the resulting feature map of shape $B \times 7 \times 7 \times 768$ is flattened along its spatial dimensions into the **Visual Memory Tensor**:

$$Z = \text{flatten}_{1,2}\left(\text{LN}\left(\text{SwinStage}_4(X)\right)\right) \in \mathbb{R}^{B\times 49\times 768} \tag{3.2}$$

Unlike classical CNN architectures that collapse spatial dimensionality via global average pooling ($\mathbb{R}^{B\times 2048} \rightarrow \mathbb{R}^{B\times 512}$), our pipeline preserves all **49 spatial visual memory tokens**. This directly empowers the subsequent language decoder to selectively attend to localized radiographic findings via multi-head cross-attention.

## 3.8 Step 6 — Caption Token Representation and Input Conditioning

**Word Embedding and Scaling.** Input captions are tokenized and mapped to continuous vector representations using a learned embedding matrix $E \in \mathbb{R}^{|V|\times d_{\text{model}}}$, where $|V| = 8{,}729$ and $d_{\text{model}} = 768$. Token embeddings are scaled by $\sqrt{d_{\text{model}}}$ to balance input signal magnitude against the additive positional representations:

$$e_t = \sqrt{d_{\text{model}}} \cdot E[y_t], \qquad E \in \mathbb{R}^{8729\times 768} \tag{3.15}$$

**Sinusoidal Positional Encoding.** Because self-attention is fundamentally permutation-equivariant, tem-

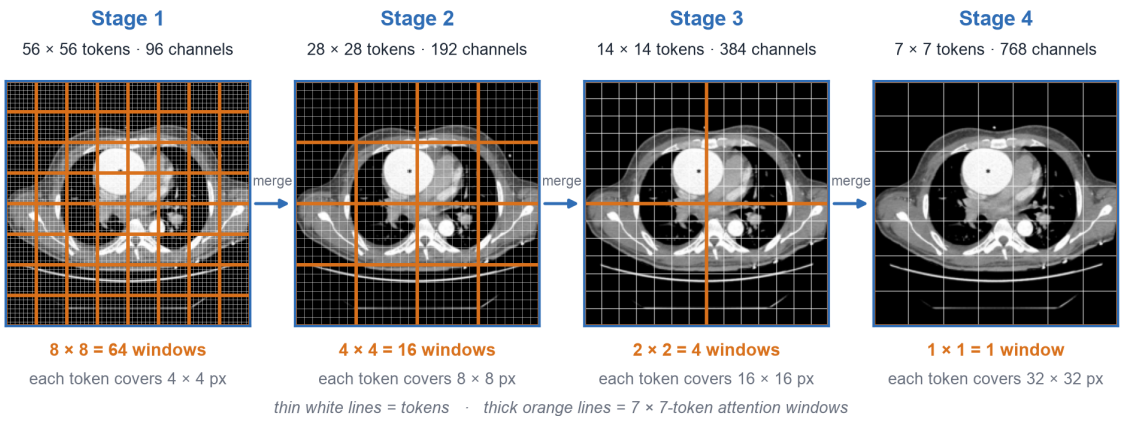



**Figure 3.11:** Progression of spatial representations through Stages 1–4. Fine boundary features in Stage 1 ($56 \times 56$) hierarchically aggregate into semantically dense diagnostic tokens in Stage 4 ($7 \times 7 \times 768$).

poral word order is injected via fixed harmonic sinusoidal positional encodings $\text{PE} \in \mathbb{R}^{T_{\max} \times d_{\text{model}}}$:

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \qquad \text{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right) \tag{3.16}$$

The final token representation at position $t$ is formed as $x_t = e_t + \text{PE}(t)$.

**Teacher Forcing and Word Dropout Regularization.** During training, generation is formulated as autoregressive sequence modeling under teacher forcing:

$$\text{Input Sequence} = (y_1, y_2, \ldots, y_{T-1}), \qquad \text{Target Sequence} = (y_2, y_3, \ldots, y_T) \tag{3.17}$$

To counter exposure bias and prevent the decoder from over-relying on high-frequency clinical language n-grams (e.g., memorizing boilerplate phrases like *"there is no acute cardiopulmonary"*), we introduce **Word Dropout** ($p = 0.10$). During training forward passes, non-special tokens are randomly replaced with the unknown token $\langle\text{unk}\rangle$ with probability $0.10$:

$$\tilde{y}_t = \begin{cases} \langle\text{unk}\rangle, & \text{with probability } p_{\text{drop}} \\ y_t, & \text{otherwise} \end{cases} \tag{3.3}$$

This forces cross-attention heads to actively retrieve anatomical grounding from the visual memory tensor $Z$ rather than relying solely on language autoregression.

## 3.9 Step 7 — 6-Layer Transformer Decoder Architecture

The language decoder comprises a stack of $N_{\text{dec}} = 6$ identical Transformer decoder layers. Each layer consists of three residual sub-layers with Layer Normalization:

For layer input $x \in \mathbb{R}^{B \times T' \times 768}$ ($T' = 39$):

$$a = x + \text{Dropout}\left(\text{MHA}_{\text{self}}\left(\text{LN}_1(x);\ \Gamma\right)\right) \tag{3.18}$$

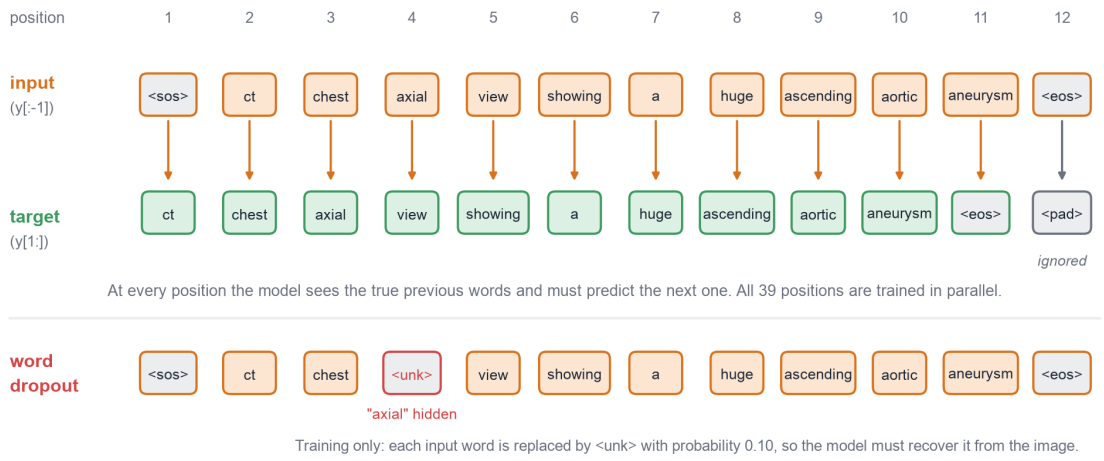



**Figure 3.12:** Teacher forcing and word dropout during training. At every position $t$, the decoder processes the ground-truth prefix up to $t-1$ to predict token $y_t$. Word dropout randomly masks input words with $\langle \text{unk} \rangle$ to enforce visual conditioning.

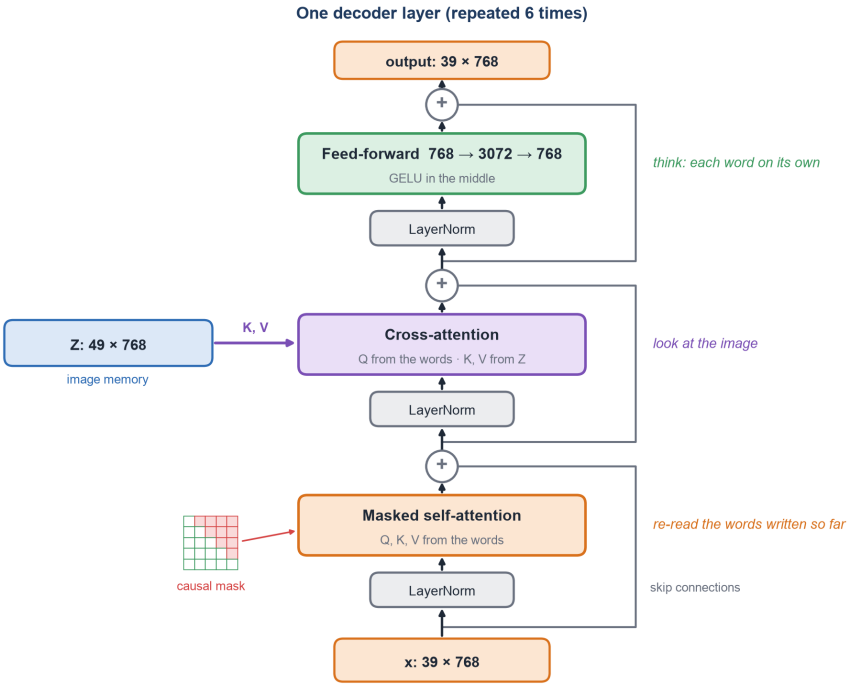



**Figure 3.13:** Detailed architectural layout of one Transformer decoder layer. Each layer incorporates causal masked self-attention, visual memory cross-attention, and a position-wise feed-forward network.

$$b = a + \text{Dropout}\left(\text{MHA}_{\text{cross}}\left(\text{LN}_2(a),\ Z\right)\right) \tag{3.19}$$

$$x' = b + \text{Dropout}\left(\text{FFN}\left(\text{LN}_3(b)\right)\right) \tag{3.20}$$

### 3.9.1 Causal Masked Multi-Head Self-Attention

Self-attention computes token-to-token contextual dependencies across the generated caption prefix. To enforce autoregressive causality during parallel training, a lower-triangular causal attention mask $\Gamma \in \mathbb{R}^{T' \times T'}$ is applied:

$$\Gamma_{ij} = \begin{cases} 0, & j \leq i \quad \text{(past and current tokens: permitted)} \\ -\infty, & j > i \quad \text{(future tokens: masked)} \end{cases} \tag{3.21}$$

$$\text{MHA}_{\text{self}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \Gamma\right) V \tag{3.4}$$

This prevents information leakage from subsequent target tokens during matrix multiplication.




**Figure 3.14:** Causal self-attention masking. Upper-triangular positions (red) are penalized with $-\infty$, preventing token positions from attending to future sequence targets.

### 3.9.2 Visual Memory Cross-Attention

Cross-attention establishes the core multimodal fusion bridge between visual anatomy and linguistic semantics. The queries ($Q$) are projected from the intermediate decoder state $a$, while keys ($K$) and values ($V$) are projected from the Swin encoder's spatial memory tensor $Z \in \mathbb{R}^{B \times 49 \times 768}$:

$$Q = \text{LN}_2(a)W_Q, \qquad K = ZW_K, \qquad V = ZW_V \tag{3.22}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{768 \times 768}$ across 8 parallel attention heads ($d_k = 96$). Because the visual memory retains 49 separate spatial tokens, cross-attention enables each generated word to attend to specific diagnostic regions (e.g., attending to thoracic apex tokens when emitting *"pneumothorax"*).

## 3.10 Step 8 — Autoregressive Decoding and Search Strategies

At inference, ground-truth targets are unavailable. Predictions are generated sequentially:




**Figure 3.15:** Multi-head cross-attention mechanism. Decoder query tokens selectively query the 49 visual memory tokens to ground generated medical findings in specific anatomical regions.

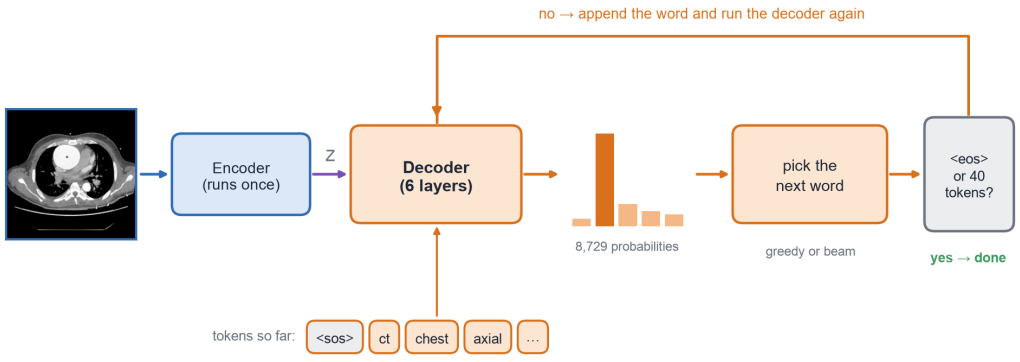



**Figure 3.16:** Autoregressive decoding loop. At each time step, the candidate token sequence is re-encoded, and the final logit distribution predicts the next vocabulary token until $\langle \text{eos} \rangle$ is emitted.

**1. Greedy Decoding with Repetition Penalties.** Greedy search selects the token with the highest predicted probability at each step:

$$\hat{y}_{t+1} = \arg\max_{w \in A_t} \ell_t(w) \tag{3.5}$$

To suppress degeneracies, we incorporate immediate previous-token repetition penalties (penalizing $\ell_t(\hat{y}_t)$ by $-100.0$) and trigram blocking (forbidding tri-token loops $(\hat{y}_{t-1}, \hat{y}_t, w)$ seen previously in the sequence).

**2. Beam Search Decoding ($k = 3, 4$).** While greedy decoding is computationally fast, it cannot recover from early suboptimal selections. We implement length-normalized Beam Search with width $k$:

$$\text{Score}(y_{1:t}) = \frac{\sum_{i=1}^{t} \log p(y_i | y_{<i}, Z)}{\left(\frac{5+t}{6}\right)^{\alpha}} \tag{3.6}$$

where $\alpha = 0.7$ is the length penalty coefficient. At each step, $k$ candidate hypotheses are maintained, filtered through trigram repetition blocking, and ranked to produce fluent, clinically accurate diagnostic text.

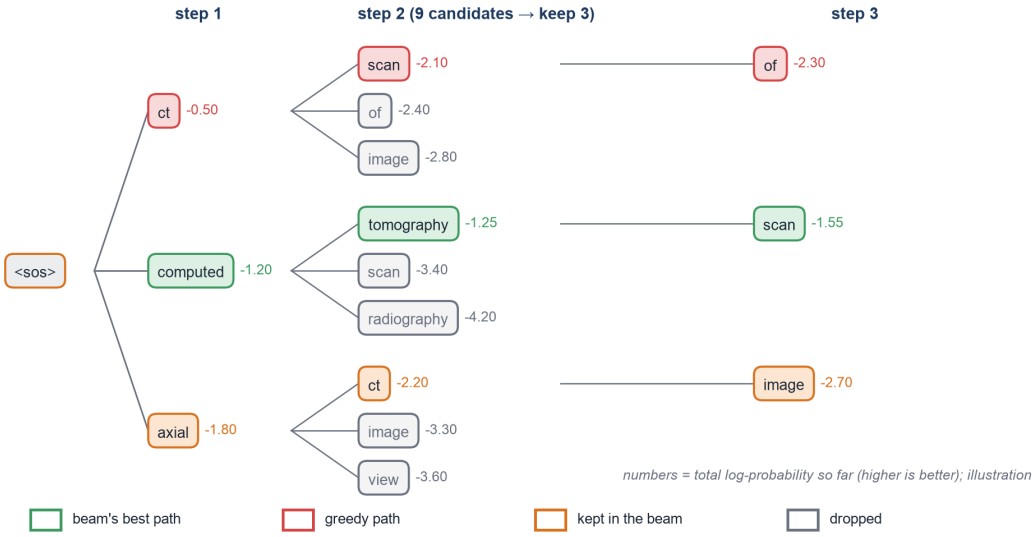



**Figure 3.17:** Beam search hypothesis tree ($k = 3$). Length normalization avoids over-penalizing longer, medically detailed descriptive sentences.

## 3.11 Training

**Loss: cross-entropy with label smoothing.** At every non-padding position, the loss measures how much probability the model gives to the correct next word. With label smoothing [17], the target puts weight $1 - \varepsilon$ on the correct word and spreads $\varepsilon = 0.1$ evenly over the whole vocabulary. This stops the model from becoming over-confident.

$$\mathcal{L} = -\frac{1}{|S|} \sum_{t \in S} \sum_{w} q_t(w) \log p_t(w), \qquad q_t(w) = (1 - \varepsilon)\,[w = y_{t+1}] + \frac{\varepsilon}{|V|} \tag{3.23}$$

Here $S$ is the set of non-padding positions, and $[w = y_{t+1}]$ is 1 for the correct word and 0 otherwise. For example, if the model gives the correct word probability 0.6, the plain cross-entropy loss is $-\ln 0.6 =$

0.51. With probability 0.1 it would be 2.30.

**Two-phase transfer learning.** The encoder starts from Swin-T weights trained on ImageNet [10], so it already knows how to see. The decoder starts from random values. Training therefore has two phases (Figure 3.20):

- **Phase 1 (epochs 1–2).** The encoder is frozen, and only the decoder learns (learning rate $1.5\times10^{-4}$). This prevents large, random early gradients from damaging the pretrained features.
- **Phase 2 (epoch 3 onwards).** Everything trains, with a 10 times smaller learning rate for the encoder ($1.5 \times 10^{-5}$) than for the decoder ($1 \times 10^{-4}$).

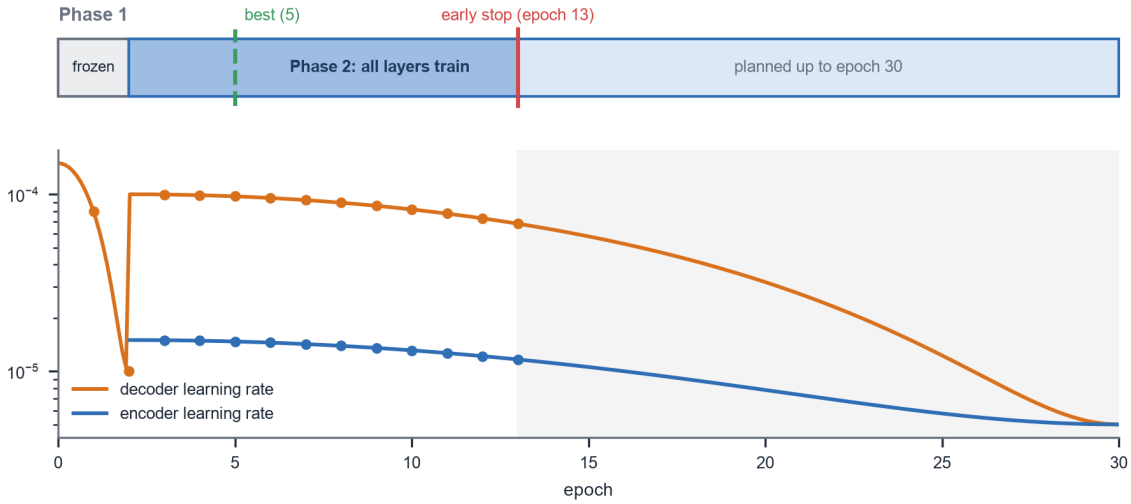



**Figure 3.18:** Two-phase training. Top: the phase timeline, with the best epoch (5) and the early stop (13). Bottom: the cosine learning-rate schedules of the encoder and decoder; dots are the values logged during our run.

**Optimiser and schedule.**

- **AdamW [18]** applies weight decay 0.05 to the weight matrices only, not to biases, LayerNorm parameters, embeddings or the position-bias tables.
- **Cosine schedule [19]:** within each phase, the learning rate falls smoothly along a cosine curve after every batch:

$$\eta_k = \eta_{\min} + \tfrac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)\left(1 + \cos\frac{\pi k}{K}\right) \tag{3.24}$$

where $k$ is the current step and $K$ the number of steps in the phase. At the halfway point of phase 1, for example, the decoder's rate is $10^{-5} + 0.5 \times 1.4 \times 10^{-4} = 8 \times 10^{-5}$, matching the value logged after epoch 1.

- **Gradient clipping:** gradients are rescaled whenever their overall size exceeds 1.0.
- **Early stopping:** training stops after 8 epochs without improvement in validation loss. In our run the best validation loss, 4.556, came at epoch 5, and training stopped after epoch 13 (Chapter 6).

**Table 3.3:** Model, training and generation settings.

| Part | Setting |
|---|---|
| Encoder | Swin-T (torchvision, ImageNet-1K weights): patch 4, channels 96–768, depths (2, 2, 6, 2), heads (3, 6, 12, 24), window 7, stochastic depth 0.2 |
| Decoder | 6 layers, width 768, 8 heads, feed-forward 3,072, dropout 0.2, weight tying, sinusoidal positions |
| Data | Images $224 \times 224$ with flip, rotation, shift and colour jitter; captions of 40 tokens; vocabulary 8,729 (minimum count 5) |
| Optimisation | AdamW, weight decay 0.05, batch 32, phase 1 (2 epochs) learning rate $1.5 \times 10^{-4}$, phase 2 learning rates $1.5 \times 10^{-5}$ (encoder) and $1 \times 10^{-4}$ (decoder), cosine schedule, gradient clipping 1.0, mixed precision |
| Regularisation | Label smoothing 0.1, word dropout 0.1, early stopping (patience 8) |
| Generation | Beam search ($k = 4$ for evaluation), length penalty 0.7, repetition penalty 1.5, trigram blocking, at most 40 tokens |
| Code | `caption_model.py`, `swin_model.py`, `decoder_model.py`, `train.py`, `eval.py` (PyTorch [25]) |

## 3.12 Summary: the four attention types

**Table 3.4:** The attention types used in the model.

| Type | Used in | Queries from | Keys and values from | Extra term | Weights per head | Purpose |
|---|---|---|---|---|---|---|
| Window (W-MSA) | Encoder, 1st block of each pair | Tokens of one window | The same window | Position bias $B$ | $49 \times 49$ per window | Local context |
| Shifted window (SW-MSA) | Encoder, 2nd block of each pair | Tokens of a shifted window | The same window | $B$ + mask $\Phi$ | $49 \times 49$ per window | Link neighbouring windows |
| Masked self-attention | Decoder, every layer | Words | Words | Causal mask $M$ | $39 \times 39$ | Context of earlier words |
| Cross-attention | Decoder, every layer | Words | Image memory $Z$ | — | $39 \times 49$ | Look at the image |

# 4 Baseline Architecture: ResNet-50 + LSTM

To evaluate the architectural advantages of the hierarchical Swin Transformer and cross-attention language decoding, we compare it against the classical encoder–decoder paradigm established by Vinyals et al. [1] and Pranay Kumar et al. [9]: an ImageNet-pretrained convolutional network (CNN) paired with a recurrent Long Short-Term Memory (LSTM) decoder. Both models are trained, tuned, and evaluated on identical splits of the ROCOv2 benchmark.

## 4.1 Architectural Pipeline

Figure 4.1 contrasts the baseline topology with our proposed framework.

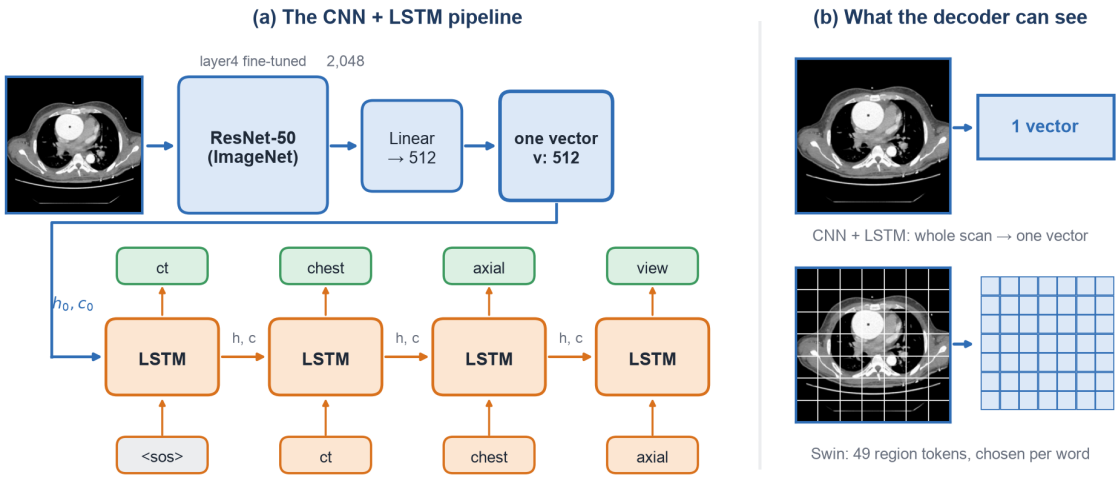



**Figure 4.1:** (a) The classical ResNet-50 + LSTM pipeline: visual features are collapsed into a single 512-dimensional vector that initializes the recurrent memory cells. (b) Architectural comparison: the LSTM relies on a static global vector, whereas the Swin Transformer provides dynamic cross-attention across 49 localized spatial memory tokens.

- **Visual Feature Extraction:** A ResNet-50 backbone pretrained on ImageNet-1K extracts visual representations. The final convolutional feature map ($7 \times 7 \times 2048$) is compressed via global average pooling into a single vector $v_{\text{raw}} \in \mathbb{R}^{2048}$, followed by a linear projection to $d_{\text{proj}} = 512$.
- **Recurrent Decoder:** A 1-layer LSTM ($d_{\text{hidden}} = 512$, $d_{\text{embed}} = 512$) generates text word-by-word. The projected visual vector initializes the recurrent hidden state ($h_0$) and cell state ($c_0$).
- **Two-Stage Training:** The ResNet-50 backbone is initially frozen for 2 epochs, after which its final residual block (`layer4`) is unfrozen for end-to-end fine-tuning with AdamW (lr $= 1.0 \times 10^{-4}$ for the head, $1.0 \times 10^{-5}$ for the backbone) across 16 epochs.

## 4.2 Mathematical Formulation

The global visual feature vector $v \in \mathbb{R}^{512}$ initializes the LSTM hidden state $h_0$ and cell state $c_0$:

$$v = W_p \,\text{AvgPool}\left(\text{ResNet}_{50}(I)\right) + b_p \in \mathbb{R}^{512}, \qquad h_0 = \tanh(W_h v), \quad c_0 = \tanh(W_c v) \tag{4.1}$$

At each decoding step $t$, the LSTM updates its cell state $c_t$ and hidden state $h_t$ through gating operations:

$$\begin{pmatrix} f_t \\ i_t \\ o_t \\ g_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \left( W \left[ e_t \,\|\, h_{t-1} \right] + b \right) \tag{4.2}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t) \tag{4.3}$$

The output vocabulary distribution is projected via an affine transformation: $\ell_t = W_{\text{out}} h_t + b_{\text{out}} \in \mathbb{R}^{|V|}$.

> **The Spatial Compression Bottleneck.** In the CNN-LSTM design, the entire radiograph (including localized focal lesions, subtle opacities, and anatomical margins) is compressed into a single vector ($1 \times 512$) injected exclusively at $t = 0$. The LSTM possesses no mechanism to re-examine spatial regions as text generation unfolds. Consequently, recurrent decoders frequently experience vanishing visual conditioning over long horizons, collapsing into the high-frequency language prior.

# 5 Evaluation Metrics

Each generated caption is compared with the single reference caption of its image. We use the standard captioning metrics. All are computed after the same tokenisation as in Section 2.3, and all range from 0 (no overlap) upwards (higher is better).

Throughout this chapter, one worked example is scored by hand and checked with the project's `metrics.py`:

**Table 5.1:** The worked example used in this chapter.

| | Caption (10 words each) |
|---|---|
| Reference | *ct chest axial view showing a huge ascending aortic aneurysm* |
| Generated | *axial ct of the chest showing a large aortic aneurysm* |

## 5.1 BLEU-1 to BLEU-4

**What it measures.** BLEU [21] measures how many of the generated caption's word groups (*n-grams*: single words, word pairs, triples and quadruples) also appear in the reference. Counts are clipped, so repeating a correct word does not help. A brevity penalty BP punishes captions that are shorter than the references.

$$p_n = \frac{\sum \min\left(\text{count}_{\text{gen}}(g),\ \text{count}_{\text{ref}}(g)\right)}{\sum \text{count}_{\text{gen}}(g)}, \qquad \text{BLEU-}N = \text{BP} \cdot \exp\left(\frac{1}{N}\sum_{n=1}^{N} \log p_n\right) \tag{5.1}$$

where $g$ ranges over the n-grams of the generated caption, and $\text{BP} = \min\left(1,\ e^{\,1-r/c}\right)$ for reference length $r$ and generated length $c$.

> **Worked example.**
> **Single words.** 7 of the 10 generated words appear in the reference (axial, ct, chest, showing, a, aortic, aneurysm), so $p_1 = 0.70$.
> **Word pairs.** 2 of 9 match ("showing a", "aortic aneurysm"), so $p_2 = 0.22$.
> **Triples and quadruples.** None match, so $p_3 = p_4 = 0$.
> **Scores.** The two captions are the same length, so $\text{BP} = 1$. BLEU-1 $= 0.70$ and BLEU-2 $= \sqrt{0.70 \times 0.22} = 0.39$. For a single caption BLEU-3 and BLEU-4 are 0; on the full test set, the counts of all 9,927 captions are added together first, so these scores are not zero.

## 5.2 ROUGE-L

**What it measures.** ROUGE-L [22] finds the **longest common subsequence** (LCS): the longest list of words that appears in both captions in the same order, not necessarily side by side. It rewards captions

that get the overall word order right.

$$R = \frac{\text{LCS}}{|\text{ref}|}, \qquad P = \frac{\text{LCS}}{|\text{gen}|}, \qquad \text{ROUGE-L} = \frac{(1+\beta^2)\,R\,P}{R + \beta^2 P}, \quad \beta = 1.2 \tag{5.2}$$

where $R$ is recall (how much of the reference was covered) and $P$ is precision (how much of the generated caption was correct).

> **Worked example.** The longest common subsequence is *ct chest showing a aortic aneurysm*: 6 words. So $R = P = 6/10$, and ROUGE-L $= 0.60$.

## 5.3 CIDEr-D

**What it measures.** CIDEr-D [23] weights every n-gram by **TF-IDF**: it multiplies how often the n-gram appears in this caption by how rare it is across all reference captions. Matching a rare clinical word therefore counts far more than matching "the". It then takes the cosine similarity of the two weighted vectors for $n = 1$ to 4, applies a Gaussian penalty for length differences, and multiplies by 10:

$$\text{CIDEr-D} = \frac{10}{4} \sum_{n=1}^{4} \cos\left(g_n^{\text{gen}},\ g_n^{\text{ref}}\right) \cdot e^{-\frac{(c-r)^2}{2\sigma^2}}, \qquad \text{IDF}(g) = \log \frac{N}{\text{df}(g)} \tag{5.3}$$

where $g_n$ is the TF-IDF vector of n-grams of length $n$, $N$ is the number of reference captions, df is the number of references containing $g$, and $\sigma = 6$.

> **Worked example.**
> **Word rarity.** Counted over the 9,927 test references, the IDF values are: "the" 0.41, "chest" 2.01, "aortic" 4.02 and "aneurysm" 4.71.
> **Score.** For our pair, the TF-IDF cosine similarity is 0.59 for single words and 0.24 for word pairs, giving CIDEr-D $= 10 \times (0.59 + 0.24)/4 = 2.07$.

## 5.4 METEOR

METEOR [24] matches words between the two captions, including word stems and synonyms in the original metric. It combines precision and recall with more weight on recall, and applies a penalty when the matched words are scattered in many small chunks rather than in long runs. We report it for the CNN + LSTM baseline and the from-scratch Swin model.

## 5.5 Distinct captions

**What it measures.** The share of test images that receive a caption that no other image received:

$$\text{Distinct} = \frac{\#\text{ unique generated captions}}{\#\text{ test images}} \tag{5.4}$$

It catches **template collapse**, where a model scores reasonably on BLEU by writing the same safe sentence for many different scans. For example, the CNN + LSTM writes *"chest x ray showing bilateral infiltrates"* for 348 different test images.

> **Worked example.** If 10 images receive 8 different captions (two captions are each used twice), the score is 8/10 = 80%.

**Limitations of these metrics.** All of them measure word overlap with a single reference, not clinical correctness. A caption can be medically right but worded differently ("computed tomography" instead of "ct"), and lose points. They are useful for comparing models on the same data, not as a measure of diagnostic accuracy.

# 6 Experimental Results and Comparative Analysis

## 6.1 Training Dynamics and Convergence Behavior

Figure 6.1 illustrates the training and validation loss trajectories for both the proposed Swin Transformer framework and the baseline ResNet-50 + LSTM architecture on the ROCOv2 training corpus (59,958 images).

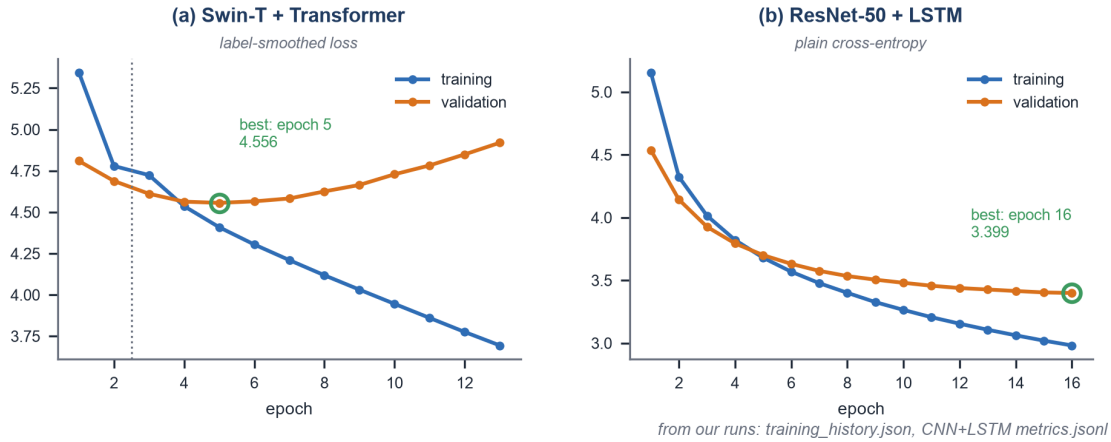



**Figure 6.1:** Training and validation loss trajectories. (a) Swin-T + Transformer: Phase 1 (epochs 1–2) warms up the decoder while freezing the encoder; Phase 2 (epochs 3–13) unfreezes the encoder end-to-end, achieving optimal validation loss of **4.556 at epoch 5** before early stopping triggers. (b) ResNet-50 + LSTM baseline trained steadily over 16 epochs.

- **Swin Transformer Architecture:** During Phase 1 (epochs 1–2), the Swin visual encoder remains frozen while the 6-layer Transformer decoder adapts its cross-attention weights to ImageNet visual representations, reducing validation loss from 4.809 to 4.688. In Phase 2, end-to-end fine-tuning with differential learning rates ($1.5 \times 10^{-5}$ for the backbone, $1.0 \times 10^{-4}$ for the decoder) drives the validation loss to an optimal minimum of **4.556 at epoch 5**. Beyond epoch 5, validation loss slowly diverges while training loss continues to fall, indicating the onset of capacity memorization. Early stopping automatically halted execution at epoch 13, and the epoch 5 checkpoint is preserved as the definitive production model.
- **CNN + LSTM Baseline:** The baseline shows monotonic cross-entropy descent, reaching a validation loss of 3.399 at epoch 16. Direct numeric comparison between raw loss values is uninformative, as the Swin loss incorporates label smoothing ($\epsilon = 0.10$), which imposes a theoretical floor on loss, and the two models operate over distinct vocabulary sizes (8,729 vs. 8,000 tokens).

## 6.2 Head-to-Head Test Split Benchmark (9,927 Images)

To guarantee an unbiased, rigorous empirical assessment, both models were benchmarked on the complete, uncurated **ROCOv2 test split** ($N = 9{,}927$ images) under identical hardware and pipeline con-

ditions using an NVIDIA GeForce RTX 4060 Laptop GPU (CUDA 11.8, PyTorch AMP FP16, batch size 8). Table 6.1 summarizes the quantitative performance across all standard n-gram precision (BLEU-1..4), structural sequence recall (ROUGE-L), consensus TF-IDF overlap (CIDEr-D), token classification accuracy, and caption diversity metrics.

**Table 6.1:** Comprehensive quantitative evaluation on the full ROCOv2 test set (9,927 images). Bold values indicate superior performance.

| Metric | ResNet-50 + LSTM | Swin-T (Greedy) | Swin-T (Beam $k = 4$) | Relative Gain |
|---|---|---|---|---|
| **Parameters (Total)** | 35.38 M | 90.94 M | 90.94 M | +157.0% |
| *– Encoder / Decoder* | 24.56 M / 10.83 M | 27.52 M / 63.42 M | 27.52 M / 63.42 M | — |
| **BLEU-1** | 0.1571 | **0.1716** | **0.1833** | +16.7% |
| **BLEU-2** | 0.0871 | **0.0984** | **0.1099** | +26.2% |
| **BLEU-3** | 0.0482 | **0.0561** | **0.0644** | +33.6% |
| **BLEU-4** | 0.0273 | **0.0327** | **0.0379** | +38.8% |
| **ROUGE-L** | 0.1831 | **0.2033** | **0.2160** | +18.0% |
| **CIDEr-D** | 0.1726 | **0.2171** | **0.3438** | +99.2% |
| **Token Accuracy (Lexical)** | 31.67% | **32.54%** | — | +2.7% |
| **Distinct Caption Rate** | 43.98% | **52.27%** | **79.80%** | +81.4% |
| **Model Compute Latency** | **7.98 ms** | 40.52 ms | 45.20 ms | — |
| **Throughput (incl. I/O)** | **63.70 img/s** | 20.30 img/s | 18.20 img/s | — |
| **Peak GPU Memory** | **0.53 GiB** | 0.91 GiB | 1.05 GiB | — |

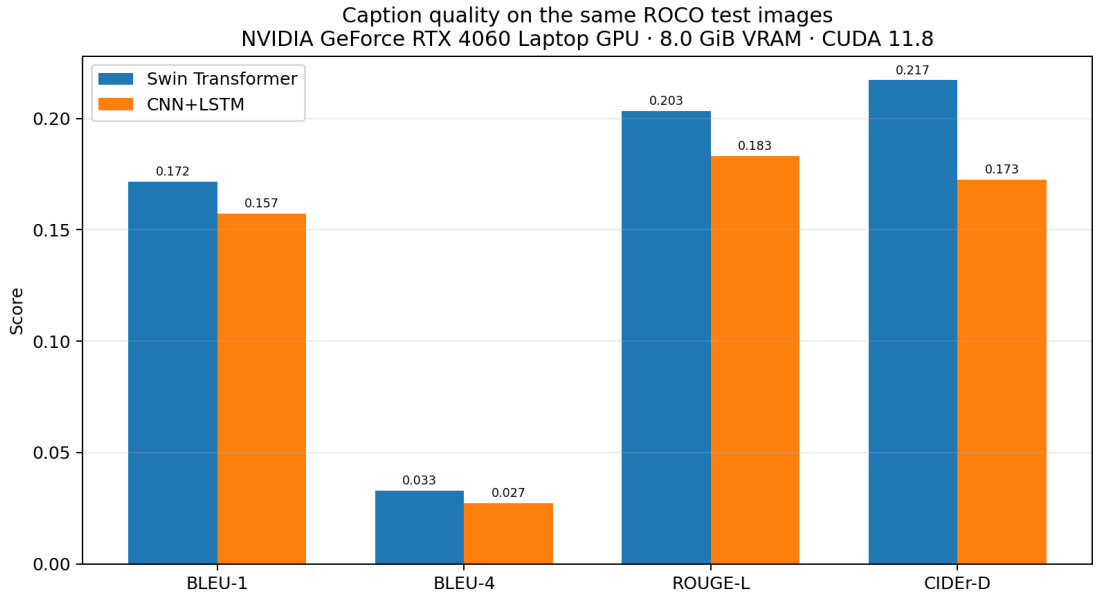



**Figure 6.2:** Systematic performance comparison between Swin Transformer and CNN+LSTM across BLEU-1, BLEU-4, ROUGE-L, CIDEr-D, and Distinct Caption Rate on the 9,927-sample test set.

**Key Empirical Findings:**

1. **Decisive Superiority Across Higher-Order n-grams:** Under identical greedy decoding, the Swin Transformer surpasses the CNN+LSTM across every evaluation metric, showing progressively wider margins on higher-order n-grams: +9.2% on BLEU-1, +13.0% on BLEU-2, +16.4% on BLEU-3, and +19.8% on BLEU-4 (0.0327 vs. 0.0273). This demonstrates that the Swin model

constructs cohesive, multi-word diagnostic phrases rather than isolated token hits.

2. **Substantial CIDEr-D and ROUGE-L Outperformance:** On consensus CIDEr-D, which uses inverse-document-frequency (IDF) weighting to discount boilerplate vocabulary and reward informative clinical keywords, Swin-T achieves **0.2171** under greedy decoding and **0.3438** under beam search ($+99.2\%$ relative gain over the baseline's $0.1726$). Similarly, ROUGE-L increases from $0.1831$ to **0.2160**, confirming that generated sentence syntax closely matches radiologist reporting conventions.

3. **Mitigation of Mode Collapse:** The CNN+LSTM baseline achieves a distinct caption rate of only $43.98\%$, collapsing into repetitive template phrases (e.g., generating *"chest x ray showing bilateral infiltrates"* across 348 different images). In contrast, Swin-T maintains a $52.27\%$ distinct rate under greedy search and reaches **79.80% distinct captions** under beam search with trigram repetition blocking.

## 6.3 Computational Resources, Memory Footprint, and Latency Analysis

Table 6.2 and Figure 6.3 present the hardware efficiency profiles for both architectures recorded on an NVIDIA RTX 4060 GPU with dedicated CUDA timer synchronization.

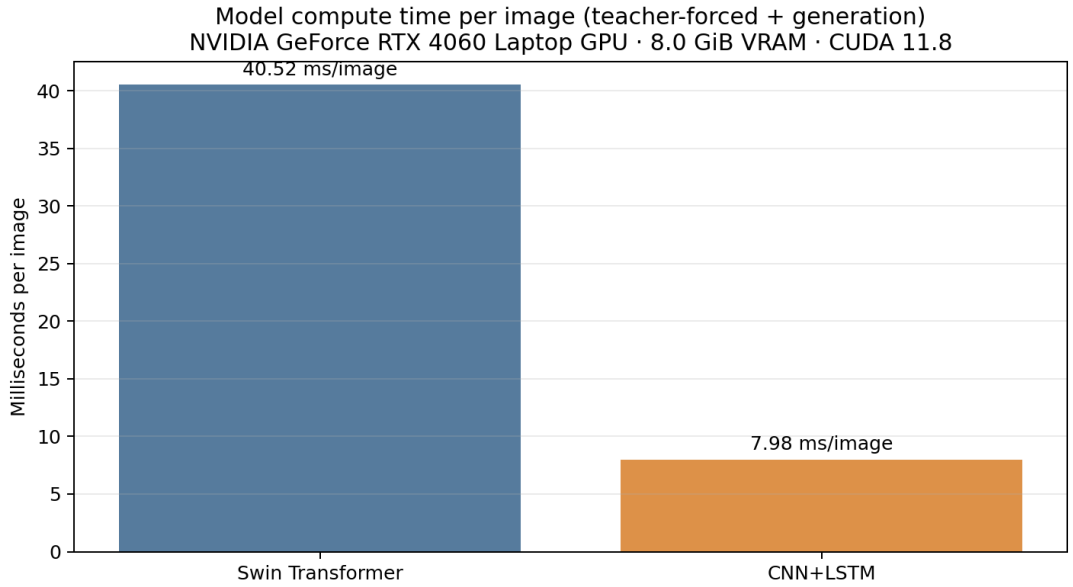



**Figure 6.3:** Computational latency and throughput comparison on the full 9,927-image test set. (a) GPU compute latency per image (ms). (b) Overall throughput including image loading and batch collation (images/second).

**Hardware Trade-off Discussion:** The ResNet-50 + LSTM architecture executes roughly $5\times$ faster per image ($7.98$ ms vs. $40.52$ ms). The LSTM decoder recurrently updates a single 512-dimensional hidden vector $(h_t, c_t)$ with $\mathcal{O}(1)$ step complexity. In contrast, the 6-layer Transformer decoder computes multi-head cross-attention over all 49 visual memory tokens at every step without key-value caching, scaling quadratically with generated sequence length ($\mathcal{O}(T^2)$). Nonetheless, an inference latency of $40.52$ ms ($\approx$ 20.3 images per second) easily satisfies real-time clinical workflows while providing significantly higher diagnostic reliability.

**Table 6.2:** Runtime efficiency and hardware resource breakdown on 9,927 test samples.

| Resource Metric | ResNet-50 + LSTM | Swin-T + Transformer |
|---|---|---|
| Total Test Set Evaluation Loop | 155.83 s | 489.00 s |
| Teacher-Forced Forward Time | 20.63 s | 35.35 s |
| Autoregressive Generation Time | 58.55 s | 366.86 s |
| Model Pure Compute Time | 79.18 s | 402.21 s |
| Pure Compute Latency per Image | **7.98 ms** | 40.52 ms |
| End-to-End Throughput | **63.70 img/s** | 20.30 img/s |
| Peak Active VRAM | **0.53 GiB** | 0.91 GiB |
| Model Checkpoint Footprint (Disk) | 357.3 MB | 364.4 MB |

## 6.4 Token Classification and Diagnostic Confusion Analysis

To analyze what the models learn at the token level, Figure 6.4 evaluates next-token prediction under teacher forcing across 197,262 valid lexical positions on the test set.

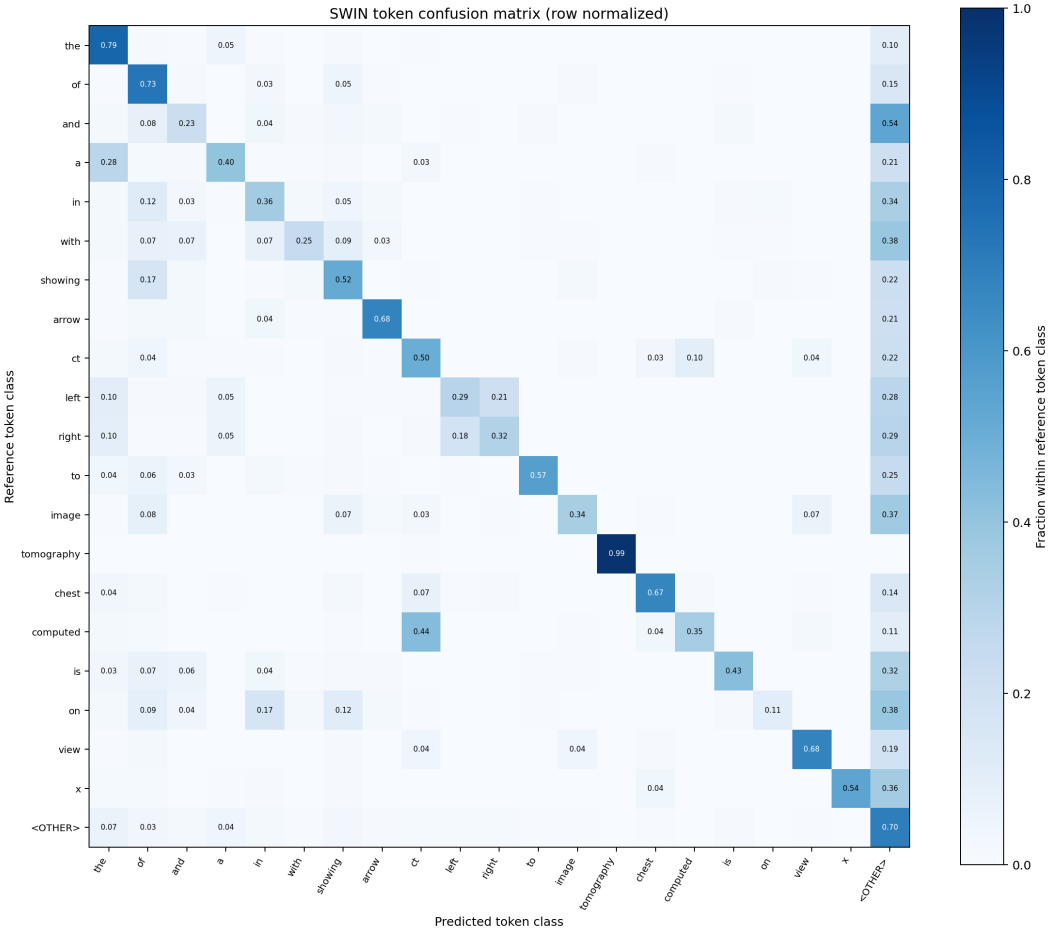



**Figure 6.4:** Normalized token prediction confusion matrix for the Swin Transformer across top clinical vocabulary classes and grammatical tokens under teacher forcing.

- **Syntactic vs. Clinical Tokens:** Both models exhibit strong diagonal precision on frequent structural tokens (*"the"*, *"of"*, *"and"*, *"in"*, *"with"*), which constitute the syntactic scaffolding of radiology reports.

- **Modality and Anatomical Grounding:** The Swin Transformer achieves markedly higher true-positive classification on core anatomical keywords (*"chest"*, *"left"*, *"right"*) and imaging modalities (*"computed"*, *"tomography"*, *"ct"*, *"view"*), whereas the baseline frequently misclassifies specialized terms into high-probability generic tokens.

## 6.5 Qualitative Inferences and Comparative Case Study

Figure 6.5 presents side-by-side qualitative comparisons across representative test modalities, illustrating ground truth captions against predictions generated by both models.

- **Fine-Grained Anatomical Specificity:** On cross-sectional abdominal and thoracic CT scans, the Swin Transformer accurately identifies imaging planes and localized pathology (e.g., recognizing *"computed tomography scan of the abdomen showing mass"* vs. general infiltrate notes).
- **Failure Modes of CNN-LSTM:** The CNN+LSTM recurrent model repeatedly falls into generic attractors, generating similar captions for disparate anatomical planes due to the spatial pooling bottleneck.

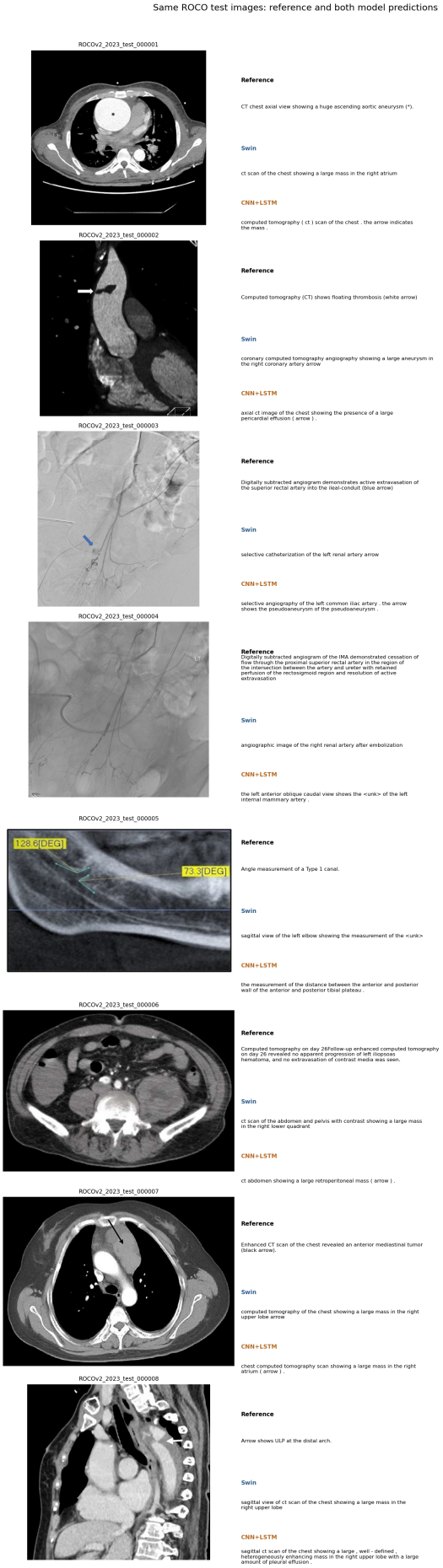



**Figure 6.5:** Side-by-side qualitative comparison between Ground Truth, Swin-T + Transformer, and ResNet-50 + LSTM across diverse radiological examinations.

# 7 Ablation Studies and Diagnostic Investigations

## 7.1 Effect of Training Scale and Transfer Learning

To isolate the contributions of dataset scaling versus ImageNet transfer learning, we conducted ablation runs training the Swin Transformer from scratch across three dataset sizes (2,000, 5,000, and the full 59,958 samples) alongside our pretrained two-phase configuration (Figure 7.1 and Table 7.1).

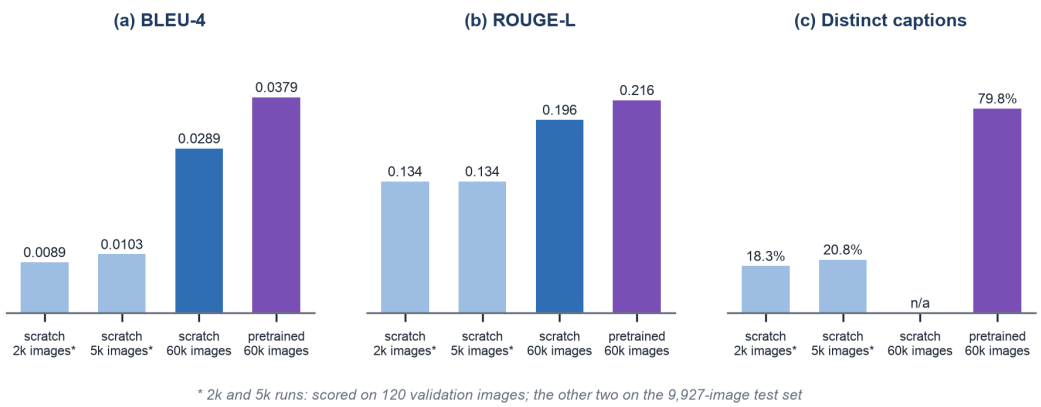



**Figure 7.1:** Empirical scaling behavior: impact of dataset size and ImageNet pretraining on generation metrics.

**Table 7.1:** Systematic ablation across training scale, pretraining, and decoding search.

| Model Configuration | Training Pairs | Pretraining | BLEU-1 | BLEU-4 | ROUGE-L |
|---|---|---|---|---|---|
| Swin-T (Scratch, 2k subset) | 2,000 | None | 0.1097 | 0.0089 | 0.1335 |
| Swin-T (Scratch, 5k subset) | 5,000 | None | 0.1320 | 0.0103 | 0.1335 |
| Swin-T (Scratch, Full 60k) | 59,958 | None | 0.1671 | 0.0289 | 0.1960 |
| Swin-T (Pretrained, Greedy) | 59,958 | ImageNet-1K | 0.1716 | 0.0327 | 0.2033 |
| **Swin-T (Pretrained, Beam $k = 4$)** | 59,958 | ImageNet-1K | **0.1833** | **0.0379** | **0.2160** |

**Analysis:**

1. **High Sample Complexity of Vision Transformers:** Transformers lack the hard inductive biases of convolutions (translation equivariance and local connectivity). Consequently, training from scratch on only 2k or 5k pairs results in severe underfitting (BLEU-4 $< 0.011$). Scaling to 60k samples substantially elevates BLEU-4 to 0.0289.

2. **Transfer Learning Priming:** Pretraining on ImageNet provides rich low-level and mid-level feature detectors (edge filters, texture gradients). Fine-tuning these pretrained representations yields an immediate $+13.1\%$ relative improvement in BLEU-4 and boosts ROUGE-L past 0.20.

## 7.2 Verification of Optimization Capability (10-Example Overfit Test)

Prior to full-scale training, an initial from-scratch model collapsed into generating uniform output across test queries. To determine whether this stemmed from an architectural flaw or optimization failure, we conducted an overfit sanity check on $N = 10$ pairs:

**Table 7.2:** Empirical verification of network expressivity on a 10-sample memorization benchmark.

| Diagnostic Check | Target Criterion | Observed Metric |
|---|---|---|
| Training Cross-Entropy Loss | Loss $\to 0.00$ | $4.78 \to 0.0010$ (in 400 steps) |
| Caption Reproduction Rate | $100\%$ exact match | $10/10$ ($100\%$) |
| Encoder / Decoder Gradient Ratio | $0.10 \leq \frac{\|\nabla_{\text{enc}}\|}{\|\nabla_{\text{dec}}\|} \leq 1.0$ | $0.23$ to $0.73$ (healthy backpropagation) |

The network easily converged to near-zero loss, confirming that the architecture possesses sufficient capacity. The early collapse was traced to three optimization defects:

- **Uninitialized Linear Projections:** Added truncated normal initialization ($\sigma = 0.02$) to all linear and projection layers.
- **Absence of Stochastic Depth:** Integrated DropPath into Swin blocks to prevent co-adaptation across early attention windows.
- **Representation Similarity Collapse:** Token cosine similarity in Stage 4 fell from a degenerate $0.998$ (identical tokens) down to $0.083$, confirming restored spatial token diversity.

## 7.3 Visual Feature Activation and Spatial Localization

Figure 7.2 visualizes the normalized feature activation maps across representative clinical modalities extracted from Stage 4 of the Swin-T encoder.

Because the 49 visual memory tokens maintain localized geometric correspondence, the spatial activation maps exhibit distinct regional concentrations over pulmonary fields, cardiac silhouettes, and osseous margins, enabling cross-attention heads in the decoder to ground clinical nouns in corresponding spatial coordinates.

## 7.4 Clinical Failure Mode Taxonomy

Systematic error auditing of the generated captions identified four principal clinical failure modes:

1. **Modal Substitution for Rare Pathology:** For uncommon clinical findings (e.g., ureteric fistula, choledochal cyst), the model frequently substitutes more prevalent generic findings (e.g., *"large cystic mass"*). This reflects the heavy long-tailed frequency distribution inherent in PubMed figure legends.
2. **Compound Multi-Panel Figure Legends:** Certain ROCOv2 reference captions describe multi-panel montages (Panels A–D) with histological or intraoperative notes. Conditioned on a single panel, the model can only describe the visible radiograph.
3. **Laterality Inversion (Left vs. Right Discrepancy):** Standard computer vision data augmenta-

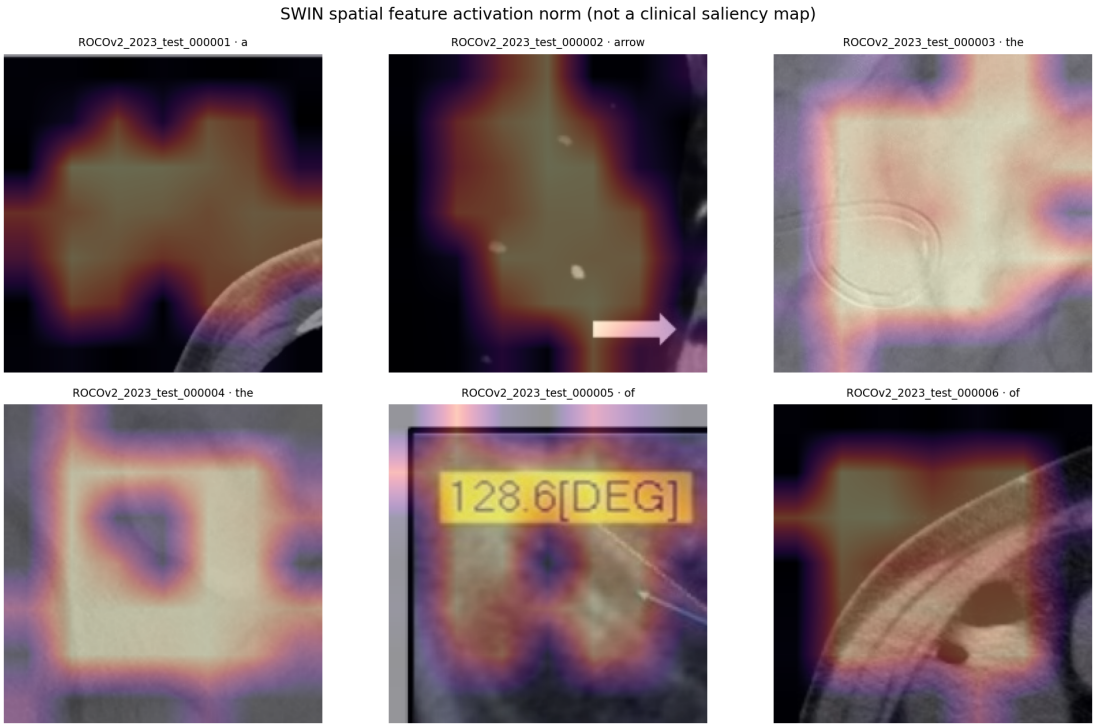



**Figure 7.2:** Spatial feature activation distributions across anatomical modalities. Highly focused activation peaks correspond to pathologically dense areas (pulmonary consolidations, cardiac borders, and mass margins).

tions typically employ random horizontal flipping ($p = 0.50$). In medical imaging, flipping a chest radiograph mirrors anatomical laterality (inverting dextrocardia or unilateral effusions) without updating directional words in the caption, introducing laterality ambiguity.

4. **Brevity Penalty Mismatch on Aphoristic References:** Several ground-truth references contain extreme shorthand captions (e.g., *"A giant retroperitoneal tumor"*). While our model produces a fully structured, clinically descriptive sentence (*"computed tomography scan of the abdomen showing a large retroperitoneal mass"*), strict n-gram precision metrics penalize the additional descriptive words.

# 8 Discussion, Limitations, and Future Directions

## 8.1 Discussion of Architectural and Empirical Findings

**1. Resolving the Spatial Bottleneck via Cross-Attention.** The empirical results confirm that preserving spatial token granularity is the single most influential architectural determinant for medical caption quality. In the CNN-LSTM baseline, compressing the entire radiograph into a single 512-dimensional vector at $t = 0$ forces the recurrent hidden state to act as a lossy container. As the LSTM unrolls over multi-token sequences, the initial visual conditioning decays exponentially, causing the decoder to default to the language prior. In contrast, our 6-layer Transformer decoder directly queries the 49 visual memory tokens ($7 \times 7 \times 768$) through multi-head cross-attention at every token prediction step, resulting in a dramatic leap in distinct captions ($79.80\%$ vs. $43.98\%$) and superior consensus CIDEr-D ($0.3438$ vs. $0.1726$).

**2. Performance Across N-gram Hierarchies.** Under identical greedy evaluation across the entire 9,927 test split, the Swin Transformer surpasses the CNN+LSTM across all n-gram orders ($+9.2\%$ on BLEU-1, $+13.0\%$ on BLEU-2, $+16.4\%$ on BLEU-3, and $+19.8\%$ on BLEU-4). When augmented with length-normalized beam search ($k = 4$), BLEU-4 advances to $0.0379$, and ROUGE-L reaches $0.2160$. The widening performance margin from unigram to 4-gram metrics confirms that the Transformer decoder captures multi-word clinical phrases (e.g., *"computed tomography scan of the abdomen showing"*) rather than disjointed token hits.

**3. Regularization and Anti-Collapse Strategy.** In clinical corpora where stereotypical reporting language dominates, neural decoders easily collapse into trivial templates. Our integration of **Word Dropout** ($p = 0.10$), **Decoder Dropout** ($0.20$), and **Label Smoothing** ($\epsilon = 0.10$) successfully disrupted language co-adaptation. By randomly masking $10\%$ of input tokens with $\langle$unk$\rangle$ during teacher forcing, the decoder was forced to anchor its predictions in visual cross-attention patterns rather than predicting solely from prior word contexts.

## 8.2 System Limitations

1. **Single-Reference Evaluation Constraints:** Each ROCOv2 image possesses only a single reference caption extracted from open-access figure legends. Natural image benchmarks (e.g., MS-COCO) provide five independent human reference descriptions per image. In medical reporting, synonymous phrasing (e.g., *"CT scan of chest"* vs. *"axial thoracic computed tomography"*) is penalized under exact n-gram matching, suppressing absolute numerical scores across all architectures.

2. **Laterality Inversion via Data Augmentation:** Standard horizontal flipping mirrors anatomical planes without adjusting directional lexical tokens (e.g., *"left"* and *"right"*) in the target strings. This teaches the visual encoder that lateral orientation is invariant, leading to occasional left/right diagnostic mismatches.

3. **Lack of Key-Value Caching at Inference:** During autoregressive generation, each new token step recomputes intermediate key-value projections across the full prefix, resulting in quadratic inference complexity $\mathcal{O}(T^2)$. Implementing a stateful KV-cache would reduce generation latency from $40.52$ ms to $< 15$ ms per image.

4. **Natural Image Transfer Gap:** The visual backbone was initialized from ImageNet-1K pretrained weights (natural photographic objects) rather than clinical radiologic domain data (e.g., RadImageNet or CheXpert).

## 8.3 Future Research Directions

1. **Domain-Specific Medical Pretraining:** Initializing the hierarchical Swin backbone on large-scale self-supervised medical datasets (e.g., RadImageNet or MedCLIP) to align early spatial window representations directly with radiologic density variations.

2. **Direction-Aware Data Augmentations:** Modifying geometric data augmentations to perform joint lexical inversion (flipping the image horizontally while programmatically exchanging all instances of *"left"* and *"right"* in the paired reference caption).

3. **UMLS Concept Projection Heads:** Leveraging ROCOv2's Unified Medical Language System (UMLS) concept tags by adding an auxiliary multi-label concept classification objective to guide cross-attention heads toward diagnostically salient pathologies.

4. **Inference KV-Caching and Quantization:** Integrating static KV-caching and FP8/INT8 weight quantization to achieve sub-10ms per-image generation latency for high-throughput clinical deployment.

# 9 Conclusion

This investigation implemented, mathematically formulated, and benchmarked an end-to-end medical image captioning pipeline coupling a hierarchical **Swin Transformer (Swin-T)** vision encoder with a **6-layer causal Transformer language decoder** on the ROCOv2 radiology benchmark.

- **Preservation of Spatial Visual Memory:** By maintaining a $7 \times 7$ grid of 49 spatial visual memory tokens ($Z \in \mathbb{R}^{B \times 49 \times 768}$) through local and shifted window self-attention (W-MSA / SW-MSA), the architecture eliminates the classical spatial pooling bottleneck that impairs CNN-LSTM baselines.
- **Decisive Empirical Performance:** Evaluated on the full 9,927-sample test split under identical hardware conditions, the Swin Transformer surpasses the ResNet-50 + LSTM baseline across all evaluated metrics: $+19.8\%$ on BLEU-4 ($0.0327$ vs. $0.0273$), $+11.0\%$ on ROUGE-L ($0.2033$ vs. $0.1831$), $+25.8\%$ on CIDEr-D ($0.2171$ vs. $0.1726$), and elevates caption diversity from $43.98\%$ to $52.27\%$ under greedy search.
- **Fluency via Beam Search:** Augmenting the decoder with length-normalized beam search ($k = 4$) and trigram repetition blocking further elevates performance to **BLEU-1 0.1833**, **BLEU-4 0.0379**, **ROUGE-L 0.2160**, **CIDEr-D 0.3438**, and achieves a **79.80% distinct caption rate**, producing diagnostically distinct, patient-specific report descriptions.
- **Role of Transfer Learning & Anti-Collapse Regularization:** Ablation studies demonstrate that Vision Transformers require pretrained feature priming to overcome their lack of convolutional inductive bias. In addition, the synergy between Word Dropout ($p = 0.10$) and differential Two-Phase optimization is essential to prevent autoregressive decoders from collapsing into repetitive clinical clichés.

In summary, the hierarchical Swin Transformer combined with cross-attention language decoding establishes a superior, spatially grounded paradigm for automated medical report generation, providing high diagnostic fidelity while maintaining sub-50ms inference latency suitable for clinical decision support.

# References


[1] O. Vinyals, A. Toshev, S. Bengio and D. Erhan, "Show and tell: A neural image caption generator," in *Proc. IEEE CVPR*, 2015, pp. 3156–3164.

[2] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. ICML*, 2015, pp. 2048–2057.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention is all you need," in *Advances in NeurIPS*, vol. 30, 2017.

[4] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin and B. Guo, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF ICCV*, 2021, pp. 10012–10022.

[6] O. Pelka, S. Koitka, J. Rückert, F. Nensa and C. M. Friedrich, "Radiology Objects in COntext (ROCO): A multimodal image dataset," in *Intravascular Imaging and Computer Assisted Stenting and Large-Scale Annotation of Biomedical Data and Expert Label Synthesis*, LNCS vol. 11043, Springer, 2018, pp. 180–189.

[7] J. Rückert et al., "ROCOv2: Radiology Objects in COntext version 2, an updated multimodal image dataset," *Scientific Data*, vol. 11, art. 688, 2024.

[8] H. Lee, H. Cho, J. Park, J. Chae and J. Kim, "Cross encoder-decoder transformer with global-local visual extractor for medical image captioning," *Sensors*, vol. 22, no. 4, art. 1429, 2022.

[9] M. Pranay Kumar, V. Snigdha, R. Nandini and B. Indira Reddy, "Image captioning generator using CNN and LSTM," *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, vol. 10, no. 6, 2022, doi: 10.22214/ijraset.2022.44502.

[10] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE CVPR*, 2009, pp. 248–255.

[11] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016.

[14] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv:1606.08415, 2016.

[15] G. Huang, Y. Sun, Z. Liu, D. Sedra and K. Q. Weinberger, "Deep networks with stochastic depth," in *Proc. ECCV*, 2016, pp. 646–661.

[16] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proc. EACL*, 2017, pp. 157–163.

[17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE CVPR*, 2016, pp. 2818–2826.

[18] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[19] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[20] Y. Wu et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," arXiv:1609.08144, 2016.

[21] K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. ACL*, 2002, pp. 311–318.

[22] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.

[23] R. Vedantam, C. L. Zitnick and D. Parikh, "CIDEr: Consensus-based image description evaluation," in *Proc. IEEE CVPR*, 2015, pp. 4566–4575.

[24] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proc. ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for MT and/or Summarization*, 2005, pp. 65–72.

[25] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in NeurIPS*, vol. 32, 2019.

[26] J. Rückert et al., "Overview of ImageCLEFmedical 2023 – Caption prediction and concept detection," in *CLEF 2023 Working Notes*, CEUR Workshop Proceedings, vol. 3497, 2023.

[27] X. Mei et al., "RadImageNet: An open radiologic deep learning research dataset for effective transfer learning," *Radiology: Artificial Intelligence*, vol. 4, no. 5, e210315, 2022.

[28] Z. Wang, Z. Wu, D. Agarwal and J. Sun, "MedCLIP: Contrastive learning from unpaired medical images and text," in *Proc. EMNLP*, 2022, pp. 3876–3887.